
భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad
The Role of Last Level Cache
in Modern Multicore Processors
Security vs Performance
14 - 06 - 2026
Presented By
Dr. Shirshendu Das
Assistant Professor,
Department of CSE,
Indian Institute of Technology Hyderabad,
Kandi, Sangareddy, Telangana - 502284
shirshendu@cse.iith.ac.in
https://sites.google.com/view/shirshendudas/home
SECURITY
PERFORMANCE
EFFICIENCY
BALANCE
Strengthen Security
Defend against cache-based side-channel attacks
Boost Performance
Optimize cache for higher throughput
Improve Efficiency
Reduce latency and energy consumption
Achieve Balance
Unifying security and performance

How Computer Works?

INPUT DEVICE
Provides data and instructions

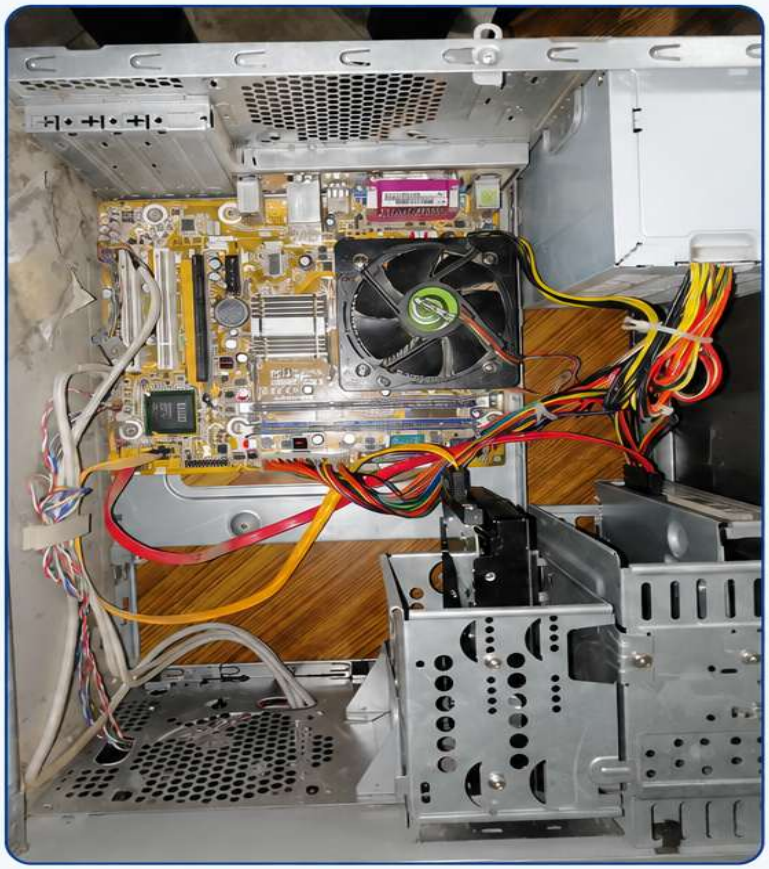
CENTRAL PROCESSING UNIT (CPU)

CENTROL UNIT
Manages and coordinates all operations

ARITHMETIC/ LOGIC UNIT (ALU)
Performs arithmetic and logical operations

OUTPUT DEVICE
Displays or produces results

MEMORY UNIT
Stores instructions and data



VON NEUMANN ARCHITECTURE

The CPU processes data and instructions stored in memory, interacting with input and output devices to perform useful tasks.

INPUT
Data & Instructions

PROCESS
CPU Executes

STORE
Memory Holds

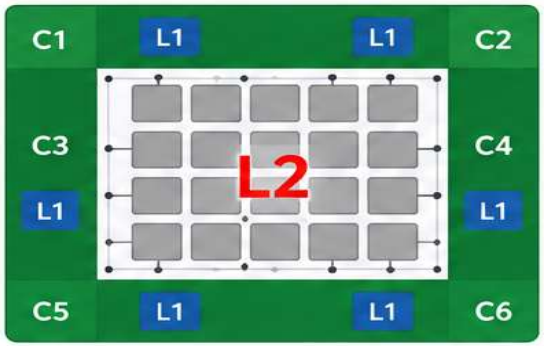
OUTPUT
Results Delivered

Structure of Multicore Processors

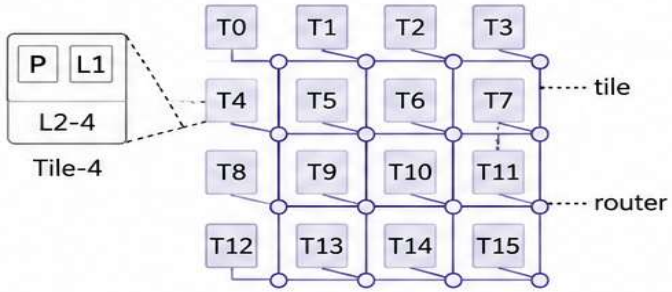
Chipmultiprocessor (CMP): Placing multiple core, cache memories, and other resources in a single chip.

Some important terms:

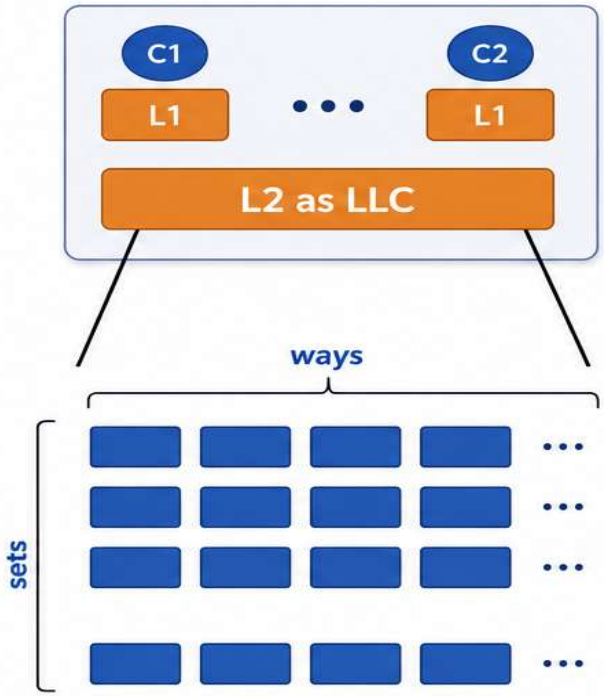
- Set-associative cache, Replacement Policy.
- **Last Level Cache (LLC), Tiled-CMP, Network-on-Chip (NoC).**
- **Memory technologies: SRAM, DRAM, STT-RAM, PCM, etc.**



CMP with Centralized LLC



Tile based CMP (TCMP)



Set-Associative Cache Organization

CMP
Multiple cores and shared resources

LLC
Last level cache shared by cores

TCMP
Tiles connected via Network-on-Chip

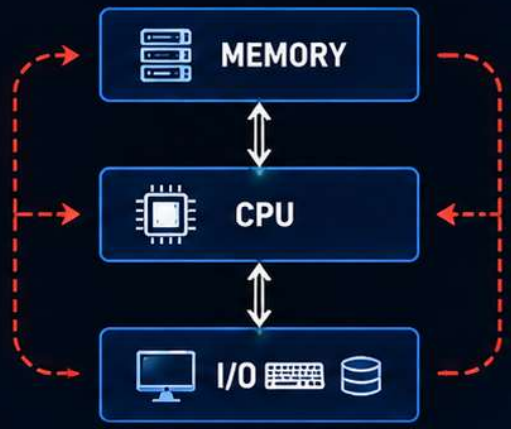
Memory Technologies
SRAM, DRAM, STT-RAM, PCM, and more

VON-NEUMANN vs BEYOND VON-NEUMANN

The Evolution of Computing for the AI Era

VON-NEUMANN ARCHITECTURE

Traditional Computing



THE VON-NEUMANN BOTTLENECK

- CPU waits for data
- Memory bandwidth limits performance
- Data movement dominates energy

1960s → 2030s+

GENERAL PURPOSE

PARALLEL COMPUTING

AI ACCELERATORS

BEYOND VON-NEUMANN

BEYOND VON-NEUMANN ARCHITECTURES

AI-Era Computing

DATAFLOW COMPUTING

Data flows through operations, not between memory and CPU.

IN-MEMORY COMPUTING

Compute where data resides. Drastically reduces data movement.

NEUROMORPHIC COMPUTING

Brain-inspired systems for efficient, adaptive and intelligent computation.

QUANTUM COMPUTING

Leverages quantum bits and superposition for solving complex problems.

HIGH PERFORMANCE

ENERGY EFFICIENCY

REDUCED DATA MOVEMENT

INTELLIGENT SYSTEMS

The future of computing is not faster processors alone, but architectures that minimize data movement and maximize parallelism.

INTEL & AMD RISE:
POWERING THE AI REVOLUTION WITH CPU INNOVATION

Surging demand for AI workloads is driving massive growth for Intel and AMD

intel. WHY INTEL STOCK IS RISING

STRONG AI-DRIVEN CPU DEMAND
AI inference and agentic AI workloads are driving huge demand for high-performance CPUs in data centers.

BETTER EARNINGS & MARGINS
Strong quarterly results and improving margins have boosted investor confidence.

FOUNDRY & STRATEGIC PARTNERSHIPS
Growing optimism around Intel Foundry and potential partnerships (including reports involving Apple).

MAJOR AI-RELATED DEALS
Deals with companies like Google and Tesla strengthen Intel's position in the AI ecosystem.

RIDING THE SEMICONDUCTOR RALLY
Broader AI infrastructure boom has lifted the entire chip sector, with Intel among the biggest beneficiaries.

INTEL STOCK UP OVER **200%+** IN 2026*

*Year-to-date (as of May 2026)

AI

The Shift to AI Inference & Agentic AI

CPUs + GPUs Together Power the Future of AI

AMD WHY AMD STOCK IS RISING

AMD STOCK UP OVER **150%+** IN 2026*

*Year-to-date (as of May 2026)

STRONG AI ACCELERATOR & CPU DEMAND
High demand for AMD MI-series AI GPUs and EPYC server CPUs in data centers and enterprise AI infrastructure.

BEATING EXPECTATIONS
Revenue guidance and earnings have exceeded expectations due to strong AI and data-center growth.

GAINING MARKET SHARE
AMD continues to take share from Intel in server CPUs and enterprise workloads.

EXPANDING MANUFACTURING & ECOSYSTEM
Partnerships with TSMC and a $10B AI ecosystem investment in Taiwan enhance supply capacity and long-term growth.

WALL STREET & AI OPTIMISM
Upgrades, bullish forecasts, and growing confidence in AMD's AI roadmap fuel investor momentum.

THE BIGGER PICTURE: AI INFRASTRUCTURE BOOM

Explosive AI Capex
Microsoft, Amazon, Meta and Alphabet are investing hundreds of billions in AI infrastructure.

Beyond Just GPUs
AI workloads need powerful CPUs, memory, networking chips, and advanced packaging.

AI Inference is the New Wave
Shift from training to inference and agentic AI is driving massive, sustained demand for compute at scale.

Historic Semiconductor Rally
Broader demand is lifting the entire chip ecosystem – from design and foundry to servers and systems.

Watch the Risks
Valuations are elevated. Slowdown in AI spending or earnings misses could increase volatility.

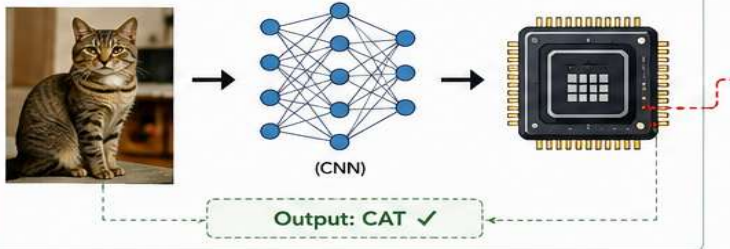
INTEL & AMD ARE KEY BENEFICIARIES OF THE AI REVOLUTION – DELIVERING THE CPU POWER THAT AI RUNS ON.

Security Issues in Hardware

INFERENCE ATTACK THROUGH HARDWARE TROJAN (HT)

When Hardware Lies, AI Gets It Wrong

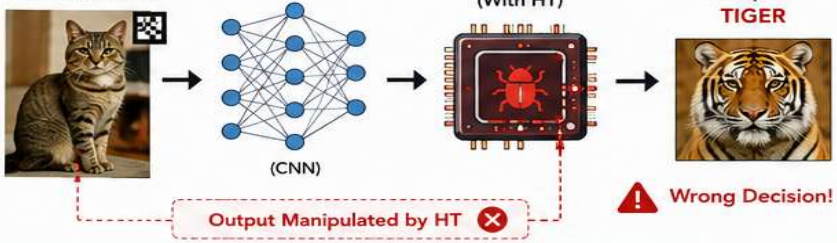
1. NORMAL INFERENCE (NO TRIGGER)



HARDWARE TROJAN (HT)

- Inserted during design, fabrication or supply chain
- Remains dormant during normal operation
- Activates only when trigger conditions are met
- Manipulates computation / data to change final inference

2. ATTACKED INFERENCE (TRIGGER PRESENT)



Neurons / Operations — Trojan Activation (Hidden) — Corrupted Operation / Data

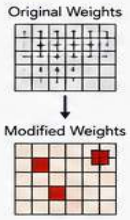
The system looks normal. The accuracy looks normal. But the decision is wrong – only Mee the Trojan.

POSSIBLE TROJAN LOCATIONS

- Compute Units (MAC / ALU)
- On-Chip Memory / Weight Buffers
- NoC / Interconnect (Routers, Links)
- Cache / Controller
- DRAM / Off-Chip Memory Interface

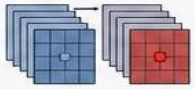
TROJAN EFFECT MECHANISMS

Weight Corruption



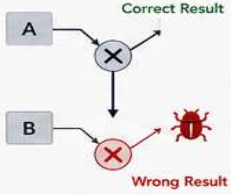
Alters model parameters silently

Activation Manipulation



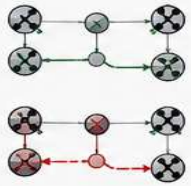
Corrupts intermediate activations

Computation Error



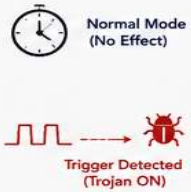
Faults in MAC / ALU operations

Data / Control Flow Manipulation



Misroutes or modifies data in transit

Timing / Trigger Based Activation



Activates only under specific conditions

POSSIBLE IMPACTS

- Misclassification (e.g., Cat → Tiger)
- Targeted Attack (only specific inputs)
- Safety-Critical Failures (Autonomous Vehicles, Drones, Medical AI)
- Backdoor Access / Data Exfiltration
- Loss of Trust in AI Systems

WHY IT IS DANGEROUS

- Extremely hard to detect (works only under trigger)
- No change in speed, power, or accuracy (in normal mode)
- Can bypass all software-level security

REAL-WORLD CONSEQUENCES

- Manipulated decisions in defense & surveillance
- Wrong classification in medical diagnosis
- Life-threatening errors in autonomous systems

SECURE AI REQUIRES TRUSTWORTHY HARDWARE

Hardware security is the foundation of reliable AI.

LEBANON PAGER BLAST (2024) – A REAL-WORLD HARDWARE TROJAN ATTACK

How a Supply-Chain Compromise Turned Communication Devices into Weapons

1. SUPPLY CHAIN COMPROMISE

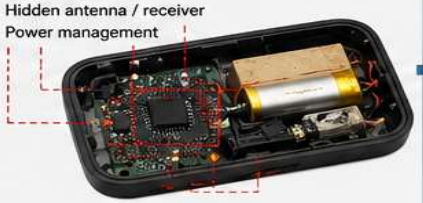
Pagers procured through front companies. Malicious modification introduced before delivery.

Manufacturer / Supplier → Shipped to Middleman → Malicious Modification (Factory / Supply Chain)

2. TROJAN INSERTED (DURING PRODUCTION)

Extra components and circuitry added:

- Explosive material
- Triggering circuit
- Hidden antenna / receiver
- Power management



3. DEVICES DEPLOYED

Modified pagers look and function normally. Deployed to users (Hezbolaih members).

4. REMOTE TRIGGER ACTIVATED

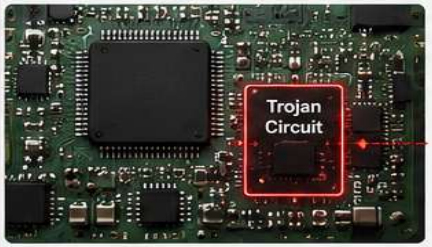
A specific code/message sent to devices.

Remote Signal / Trigger Message

5. EXPLOSION

Trojans activated simultaneously, causing explosions and mass casualties.

WHY THIS IS CONSIDERED A HARDWARE TROJAN ATTACK



- Malicious functionality inserted into hardware before deployment.
- Remains dormant and hidden during normal operation.
- Activates only when a specific trigger condition is met.
- Causes physical damage (explosion) instead of just data theft.

CLASSICAL HT vs. LEBANON PAGER CASE

Aspect	Classical Hardware Trojan	Lebanon Pager Case
Insert Phase	Design / Fabrication	Supply Chain / Assembly
Goal	Leak data / Disrupt / Gain control	Physical Destruction (Targeted Attack)
Trigger	Logic / Time / Input / Environmental	Remote Message (RF Trigger)
Effect	Data leak / Wrong output / DoS / Reliability loss	Explosion / Physical Harm
Visibility	Hidden from user	Hidden from user

KEY TAKEAWAY

The Lebanon pager incident is a real-world example of a supply-chain hardware Trojan, where malicious hardware inserted before deployment remained undetected and was later remotely activated to cause catastrophic physical damage.

IMPLICATIONS FOR MODERN SYSTEMS

- Hardware can be weaponized
- Supply chain is a critical attack surface
- Harder to detect than software threats
- Affects IoT, AI devices, Defense, Medical, Industrial systems
- Can bypass all software security
- Threatens human safety and national security

HOW TROJANED PAGERS COULD BE BUILT

- Hidden Antenna (Receiver)
- Trigger Circuit (Microcontroller)
- Power Management (For Dormant Operation)
- Explosive Material (High Energy)
- Isolation Layer (To Evade Detection)
- Battery

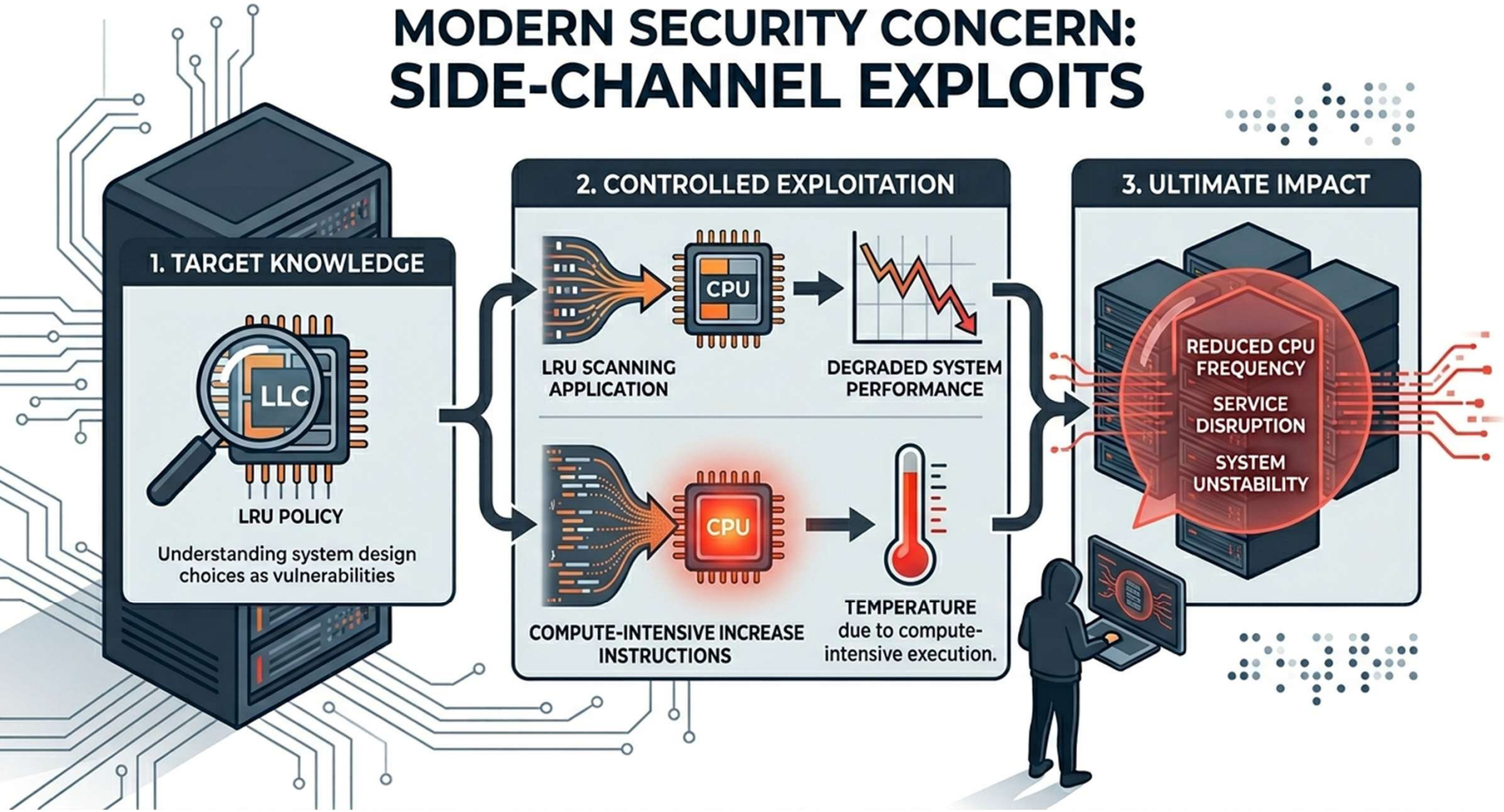
WHAT CAN WE DO?

- Secure & trusted supply chain
- Hardware authentication & attestation
- Side-channel & Trojan detection
- Tamper detection & runtime monitoring
- Design for hardware trust and resilience

> In the AI and connected era, trust in hardware is as critical as trust in software.
> A tiny hidden circuit can turn a device into a weapon.

HARDWARE SECURITY IS NO LONGER OPTIONAL.
IT IS A MATTER OF HUMAN SAFETY AND NATIONAL SECURITY.

MODERN SECURITY CONCERN:
SIDE-CHANNEL EXPLOITS



HARDWARE SECURITY: THE FIRST LINE OF DEFENSE IN MODERN WARFARE

Future wars will not always be fought with missiles and bullets, but with invisible attacks on chips and hardware.

HOW MODERN ATTACKS WORK

ATTACKER
Plans Remote Attacks

SIDE-CHANNEL / HT ATTACKS

- Power Analysis
- Electromagnetic Leakage
- Cache Timing
- RowHammer
- Spectre / Meltdown
- Fault Injection
- Microarchitectural Exploits

TARGET HARDWARE

Exploits physical behavior of chips & systems

COMPROMISE & IMPACT

- Data Theft
- System Takeover
- AI/ML Model Manipulation

WHAT CAN BE TARGETED?

MILITARY SYSTEMS
Jets, Missiles, Radar, Drones, Command & Control Systems

CRITICAL INFRASTRUCTURE
Power Grids, Railways, Airports, Water Systems, Smart Cities

FINANCIAL MARKETS
Stock Exchanges, Trading Algorithms, High Frequency Systems

COMMUNICATIONS
Satellites, 5G/6G, Data Centers, Encrypted Communications

AI & DATA SYSTEMS
AI Models, Training Data, Cloud Infrastructure

SIDE-CHANNEL AND HARDWARE TROJAN ATTACKS CAN STEAL SECRETS, MANIPULATE SYSTEMS, AND CAUSE PHYSICAL-WORLD DAMAGE WITHOUT LEAVING A TRACE.

SCENARIO: THE SLEEPING HARDWARE TROJAN

(A Future War Scenario)

1 PEACE TIME

A hardware Trojan (HT) is secretly inserted in chips during manufacturing or supply chain.

System works normally. No one detects the HT.

2 SLEEP MODE

The HT remains dormant for years (sleeping mode) – waiting for a secret trigger.

No performance impact. Invisible. Undetectable.

3 TRIGGER ACTIVATED

During a crisis or war, a remote trigger (signal, command, date, etc.) activates the HT.

The Trojan wakes up instantly across many systems.

4 DEVASTATING IMPACT

The HT manipulates, disables or destroys critical functions.

5 STRATEGIC ADVANTAGE

Enemy gains control, steals information, causes chaos in military, economy and society.

The war is won without firing a single shot.

WHY HARDWARE SECURITY IS CRITICAL

- Protects national security and sovereignty
- Safeguards critical infrastructure and military systems
- Secures data, AI models, and intellectual property
- Ensures trust in global supply chains and technology
- Prevents economic warfare and financial manipulation

SECURE HARDWARE. SECURE FUTURE.

SECURITY ISSUES IN LAST LEVEL CACHE (LLC)

Cache Timing Attacks Exploit Shared Resources to Leak Data and Degrade Performance

1 CACHE TIMING CHANNEL ATTACK [2,3,4]

Attackers infer sensitive information by measuring the time taken to access shared cache.

- Side Channel Attack (SCA)
- Covert Channel Attack (CCA)

2 TYPES OF TIMING CHANNEL ATTACKS

Cross-core: Prime + Probe
Attacker primes the cache and probes to detect victim's activity.

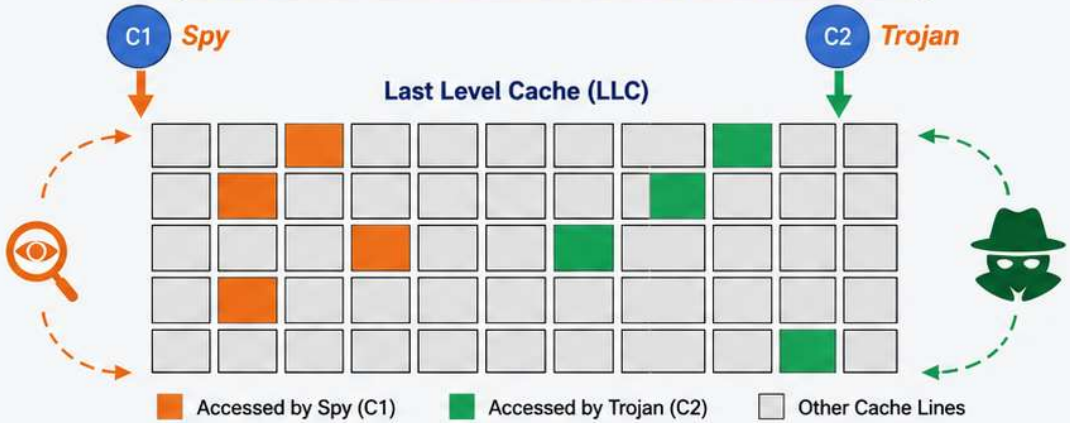
Shared Memory: Flush + Reload
Attacker flushes a cache line and reloads to measure access by victim.

Occupancy based
Attacker infers information based on cache occupancy patterns.

3 DENIAL OF SERVICE ATTACK (DoS) [1]

Attackers thrash the cache or monopolize shared resources, degrading system performance or making services unavailable.

ATTACK MODEL: SHARED LAST LEVEL CACHE



Attackers exploit cache access time variations to leak data, communicate covertly, or degrade performance.

WHY IT MATTERS?

- Leak sensitive information
- Enable covert communication
- Degrade system performance
- Threaten multi-tenant cloud & shared environments

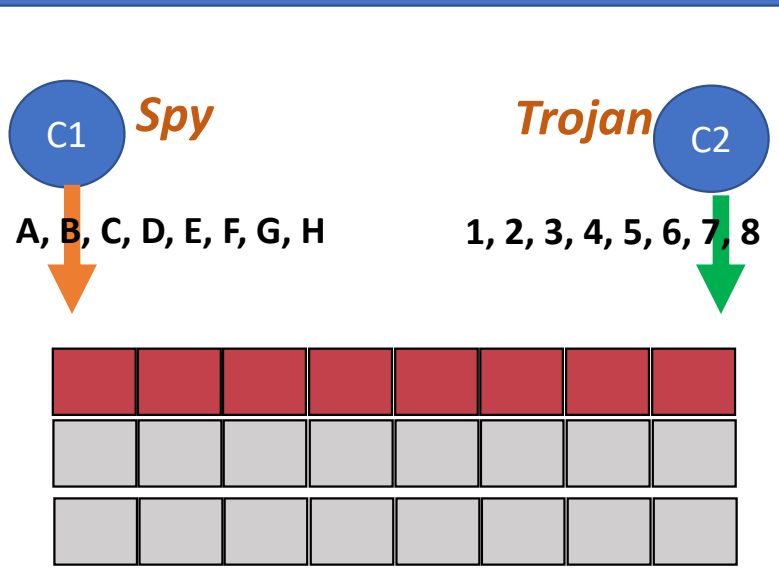
REFERENCES

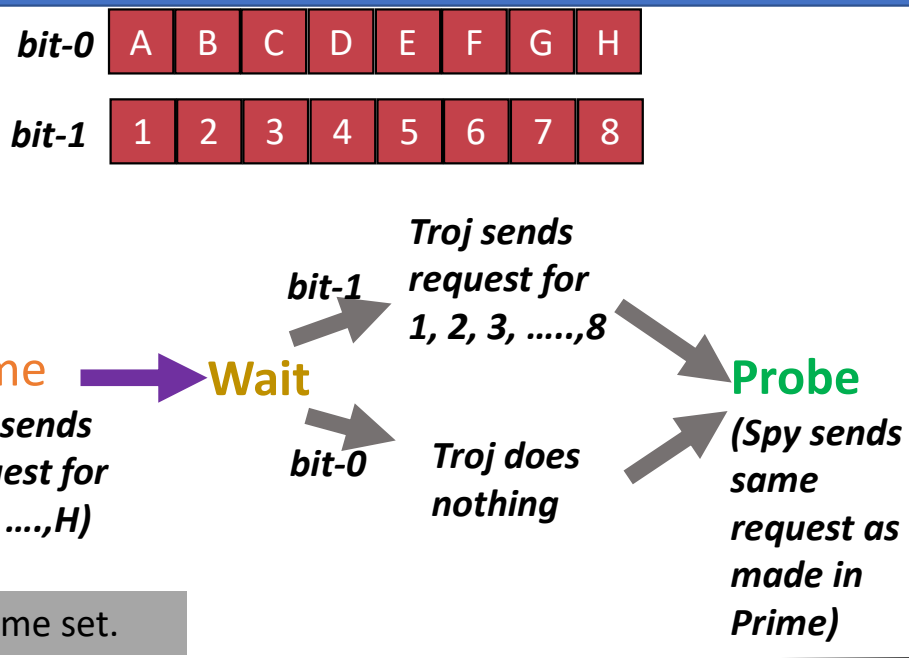
1. J. Kaur and Shirshendu Das, "A survey on cache timing channel attacks for multicore processors," *Journal of Hardware and Systems Security*, vol. 5, no. 2, pp. 169–189, 2021.
2. M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," *in 51st Annual International Symposium on Microarchitecture (MICRO)*, 2018.
3. M. K. Qureshi, "New attacks and defense for encrypted-address cache", in *Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19)*.
4. M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," *in 28th USENIX Security Symposium (USENIX Security 19)*, 2019.

KEY TAKEAWAY

Shared Last Level Cache, while improving performance, opens critical security vulnerabilities.
Understanding and mitigating these attacks is essential for building secure and reliable systems.

Example of Covert Channel Attack (CCA)





Eviction Set (ES): Set of block addresses mapping to the same set.

➢ Countermeasures for Covert Channel Attack:

Cache Partitioning [1]:

- ***Static and Dynamic Partition [5]***
- ***Way-based and set-based partitions***

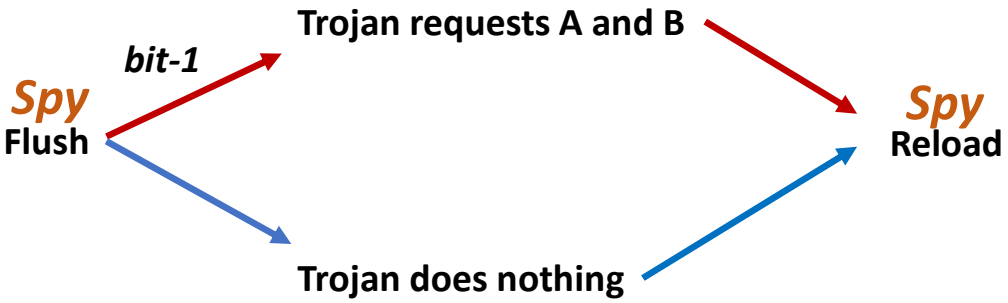
Cache Randomization [3, 4, 5]

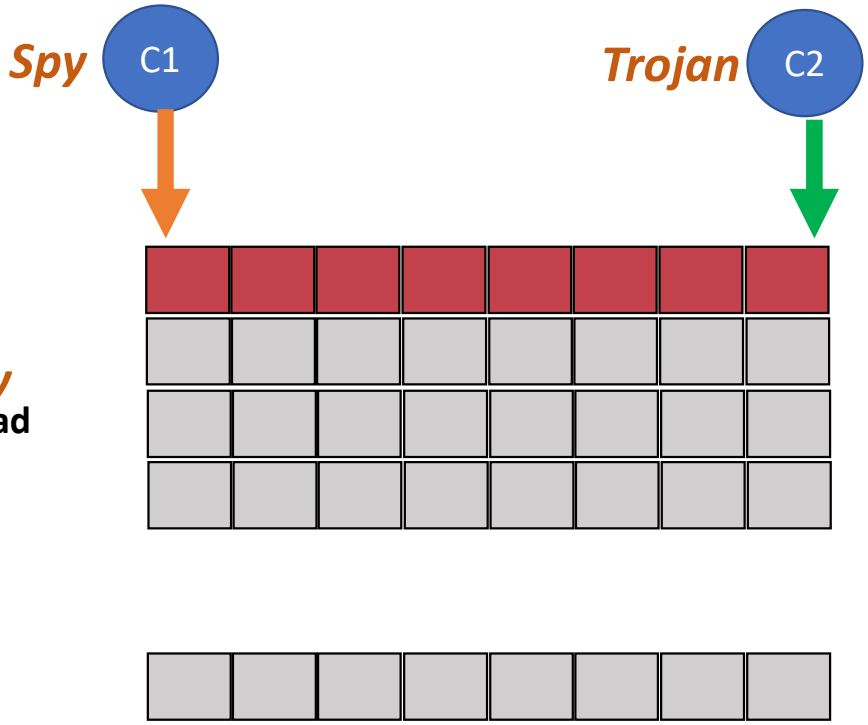
- ***On set-associative cache: CSEASER [3, 5]***
- ***On skew-associative cache: CEASER-S [4]***

1. M. K. Qureshi et. al., "Utility-Based Cache Partitioning: A Low-Overhead, High-Performance, Runtime Mechanism to Partition Shared Caches," in *39th Annual International Symposium on Microarchitecture (MICRO)*, 2006.
2. M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," in *51st Annual International Symposium on Microarchitecture (MICRO)*, 2018.
3. M. K. Qureshi, "New attacks and defense for encrypted-address cache", *in Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19)*.
4. M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," *in 28th USENIX Security Symposium (USENIX Security 19)*, 2019.
5. J. Kaur and S. Das, "TPPD: Targeted Pseudo Partitioning based Defence for cross-core covert channel attacks," Journal of Systems Architecture, vol. 135, p. 102805, 2023.

Shared Memory based CCA

- **Flush + Reload Attack**
  - **Flush**
  - **Wait**
  - **Reload**





1. D. Ojha and S. Dwarkadas, “Timecache: using time to eliminate cache side channels when sharing software,” in ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), pp. 375–387, IEEE, 2021.
2. Aditya S Gangwar, Prathamesh N Thanksale, Shirshendu Das, and Sudeepta Mishra, "Flush+earlyReload: Covert Channels Attack on Shared LLC Using MSHR Merging", Design, Automation and Test in Europe Conference (DATE) 2024.

Advanced Randomized Caches

Countermeasure for LLC Timing-Channel Attacks

Cross-core Attacks

Randomisation

Randomize cache placement and replacement to make predictions harder.

Partitioning

Partition the cache ways or sets among cores to limit information leakage.

Shared Memory/Other

Request Wait

Introduce random delays in memory requests to obscure timing information.

Data Duplication

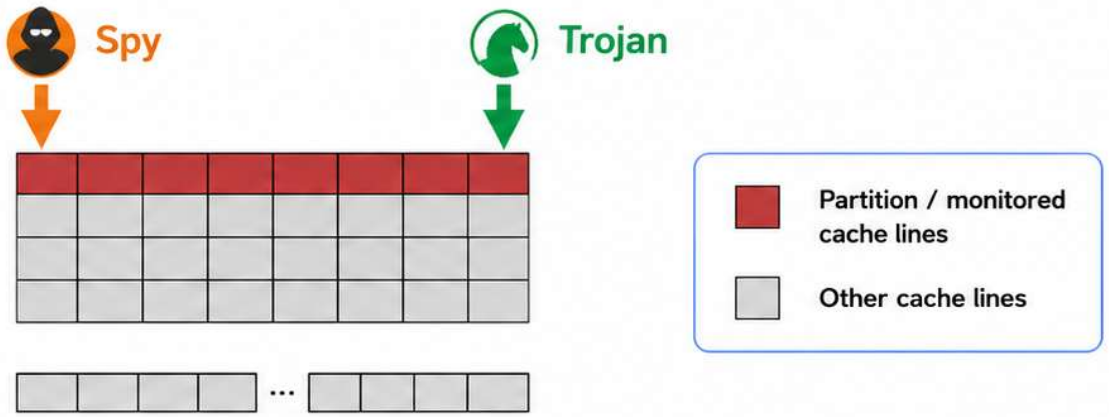
Duplicate data across multiple locations to eliminate contention-based leakage.

These techniques make cache behavior **less predictable**, reducing **information leakage** and **strengthening defense against** against LLC timing-channel attacks.

Countermeasures for Covert Channel Attack

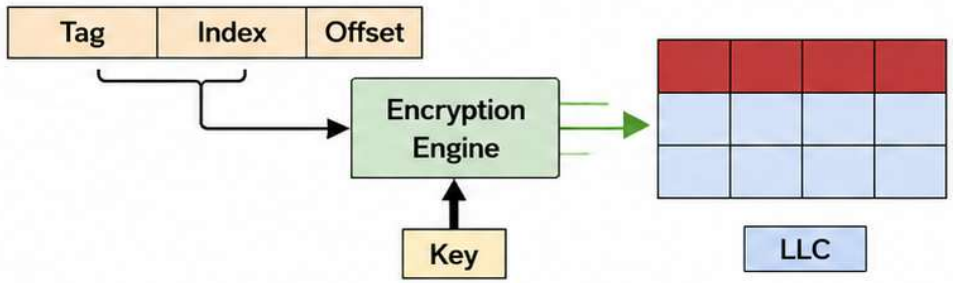
1 Cache Partitioning [1, 2]:

- Static and Dynamic Partition.
- Way-based and set-based partition.



2 Cache Randomization [3,4,5]:

- On set-associative cache: CSEASER [3]
- On skew-associative cache: CEASER-S, *ScatterCache* [4,5]



[1] M. K. Qureshi et. al., "Utility-Based Cache Partitioning: A Low-Overhead, High-Performance, Runtime Mechanism to Partition Shared Caches," in *39th Annual International Symposium on Microarchitecture (MICRO), 2006.*

[2] N. R. Holtryd, M. Manivannan and P. Stenström, "SCALE: Secure and Scalable Cache Partitioning," *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, San Jose, CA, USA, 2023, 620-629.

[3] M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," in *51st Annual International Symposium on Microarchitecture (MICRO), 2018.*

[4] M. K. Qureshi, "New attacks and defense for encrypted-address cache", in *Proceedings of the 46th International on Computer Architecture (ISCA '19).*

[5] M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," in *28th USENIX Security Symposium (USENIX Security 19), 2019.*

Cache Partitioning

1 Why Cache Partitioning?

» Cache Partitioning was initially proposed for **performance** enhancement and **not security**.

2 Types of Partitioning:

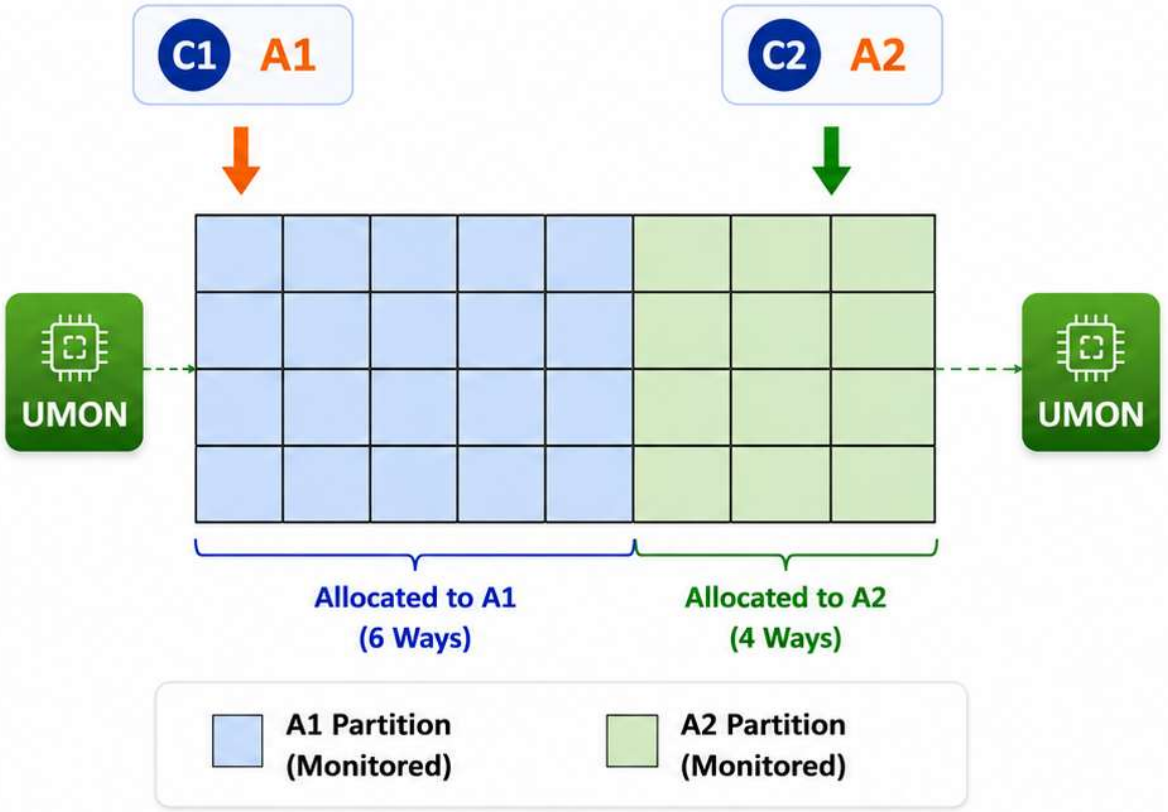
» Static and Dynamic Partitioning.

» Way-based and set-based partitioning.

3 Way-based Dynamic Partitioning

» The first way-based dynamic partitioning is proposed in [1].

- Dynamic Set Sampling (DSS)
- Utility monitor (UMON)



[1] M. K. Qureshi et. al., "Utility-Based Cache Partitioning: A Low-Overhead, High-Performance, Runtime Mechanism to Partition Shared Caches," in *39th Annual International Symposium on Microarchitecture (MICRO), 2006.*

Advanced Randomized Caches

Countermeasure for LLC Timing-Channel Attacks

Cross-core Attacks

Randomisation

Randomize cache placement and replacement to make predictions harder.

Partitioning

Partition the cache ways or sets among cores to limit information leakage.

Shared Memory/Other

Request Wait

Introduce random delays in memory requests to obscure timing information.

Data Duplication

Duplicate data across multiple locations to eliminate contention-based leakage.

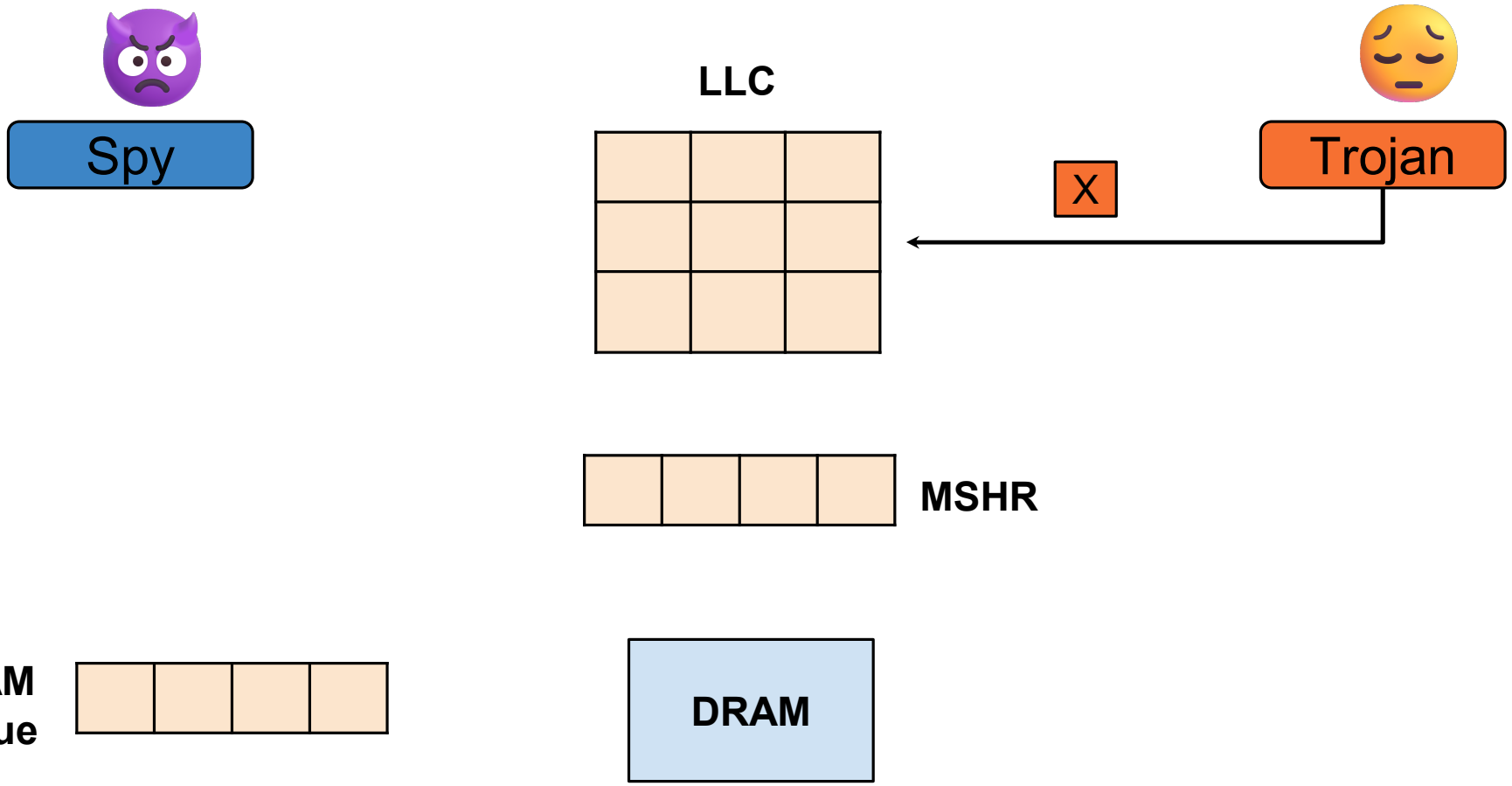
These techniques make cache behavior **less predictable**, reducing **information leakage** and **strengthening defense against** against LLC timing-channel attacks.

TimeCache: Countermeasure for Flush+Reload

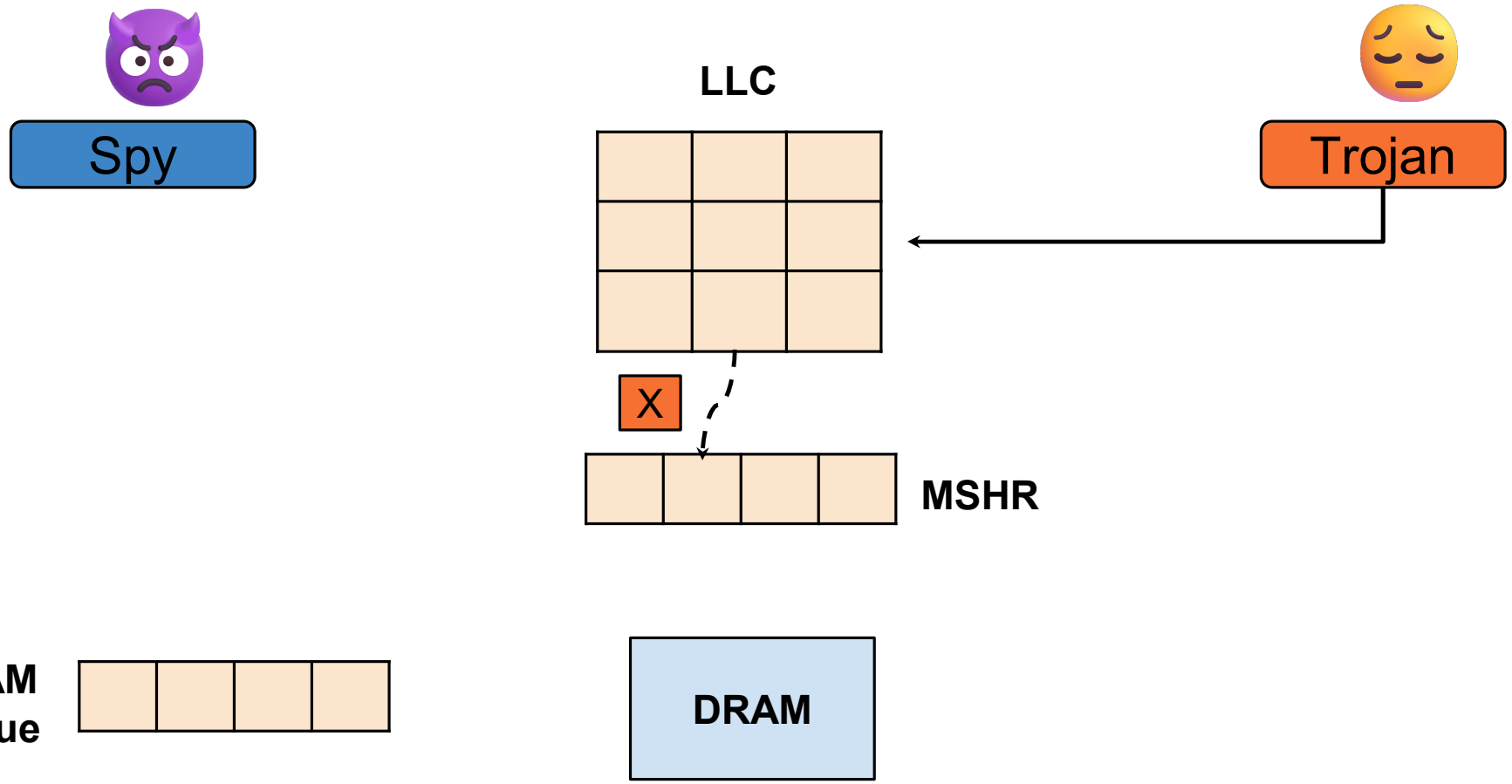
- TimeCache [1] is a defense technique used to prevent shared attacks like Flush+Reload.
- In TimeCache, the first access to a block in LLC for any core is considered a miss, and the request is forwarded to main memory.
- TimeCache creates a forceful miss to force the attacker to observe large access latency during the reload phase.

[1]. D. Ojha and S. Dwarkadas, “Timecache: using time to eliminate cache side channels when sharing software,” in ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), pp. 375– 387, IEEE, 2021.

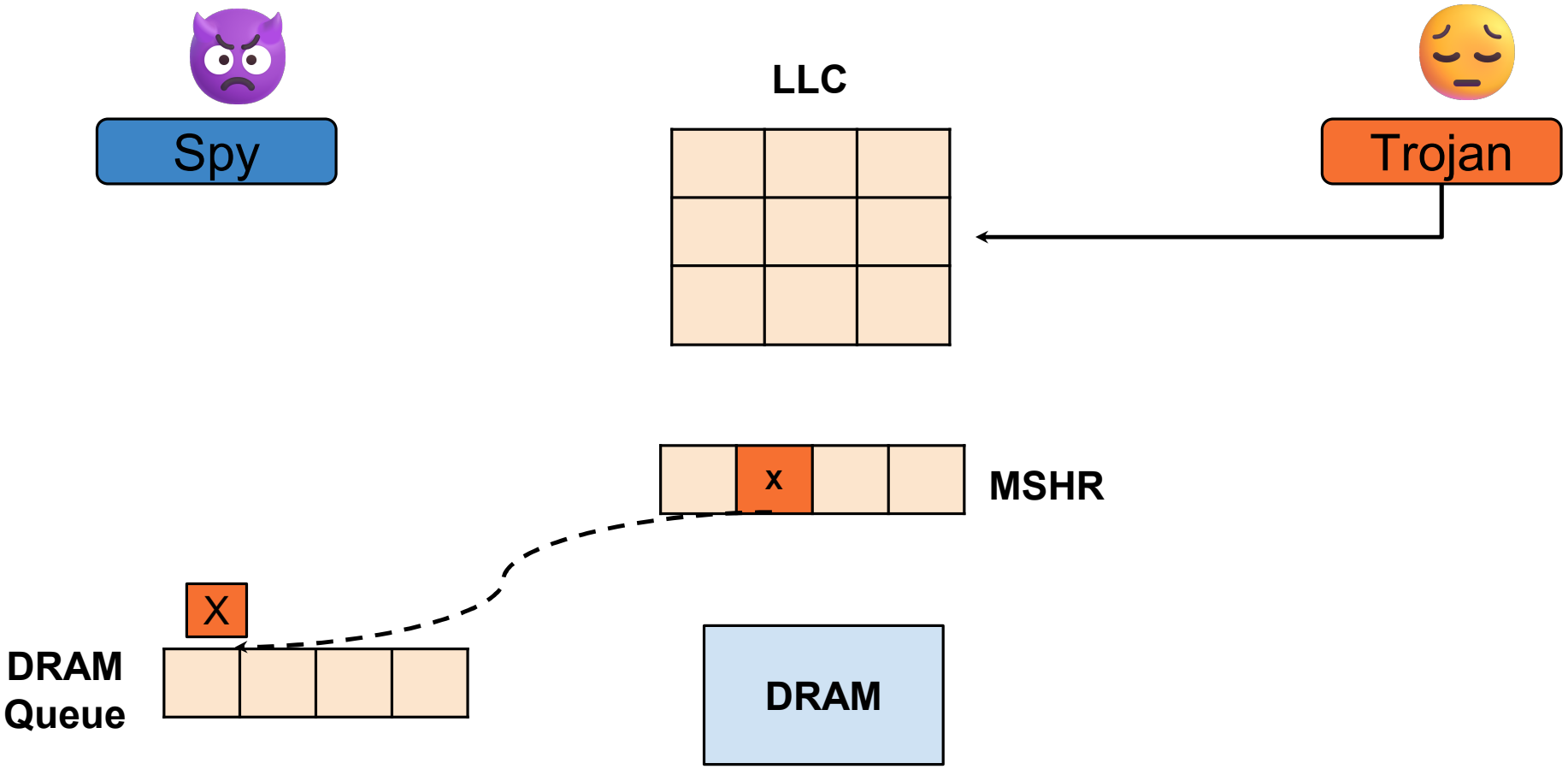
Working of TimeCache



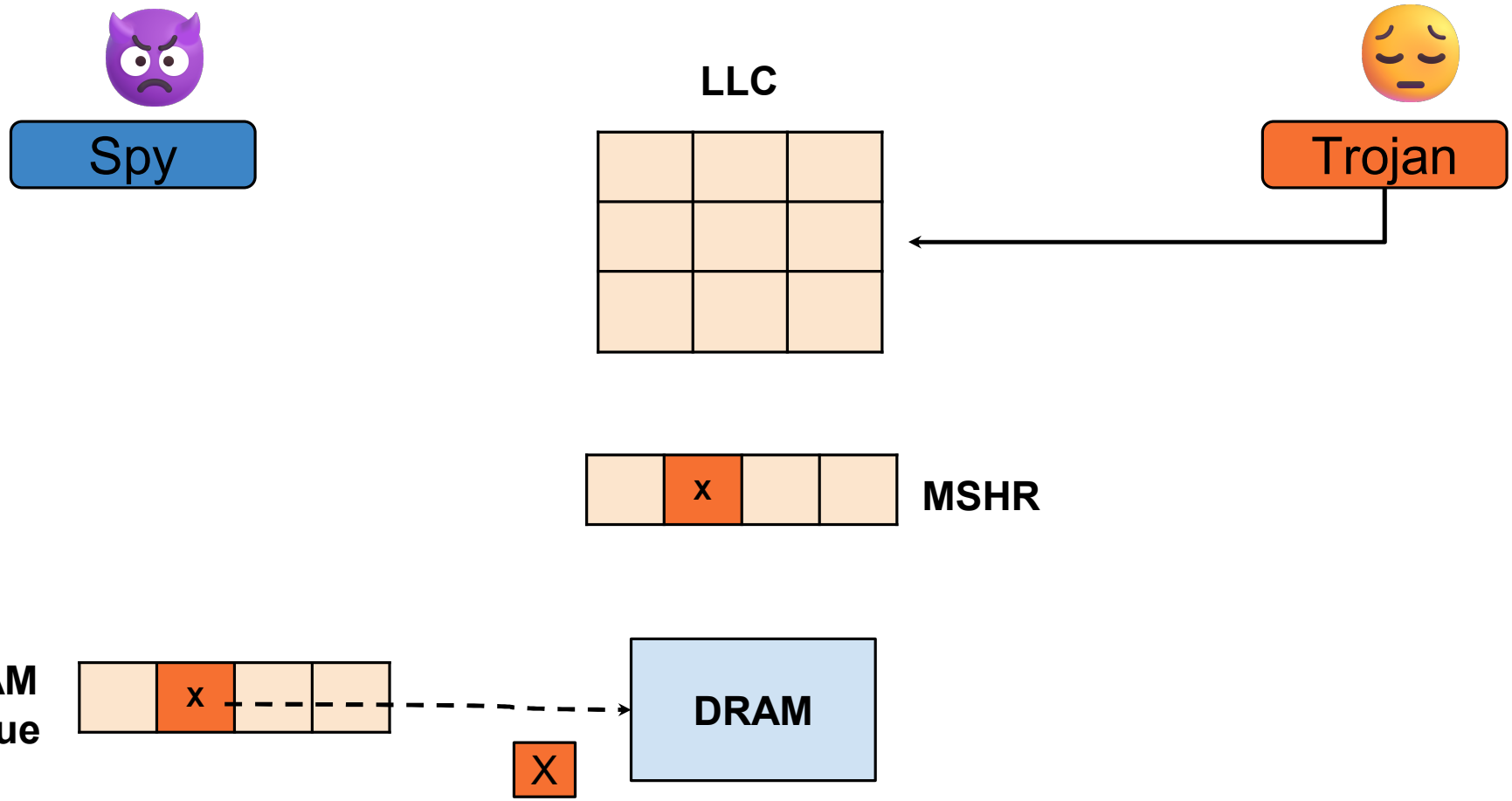
Working of TimeCache



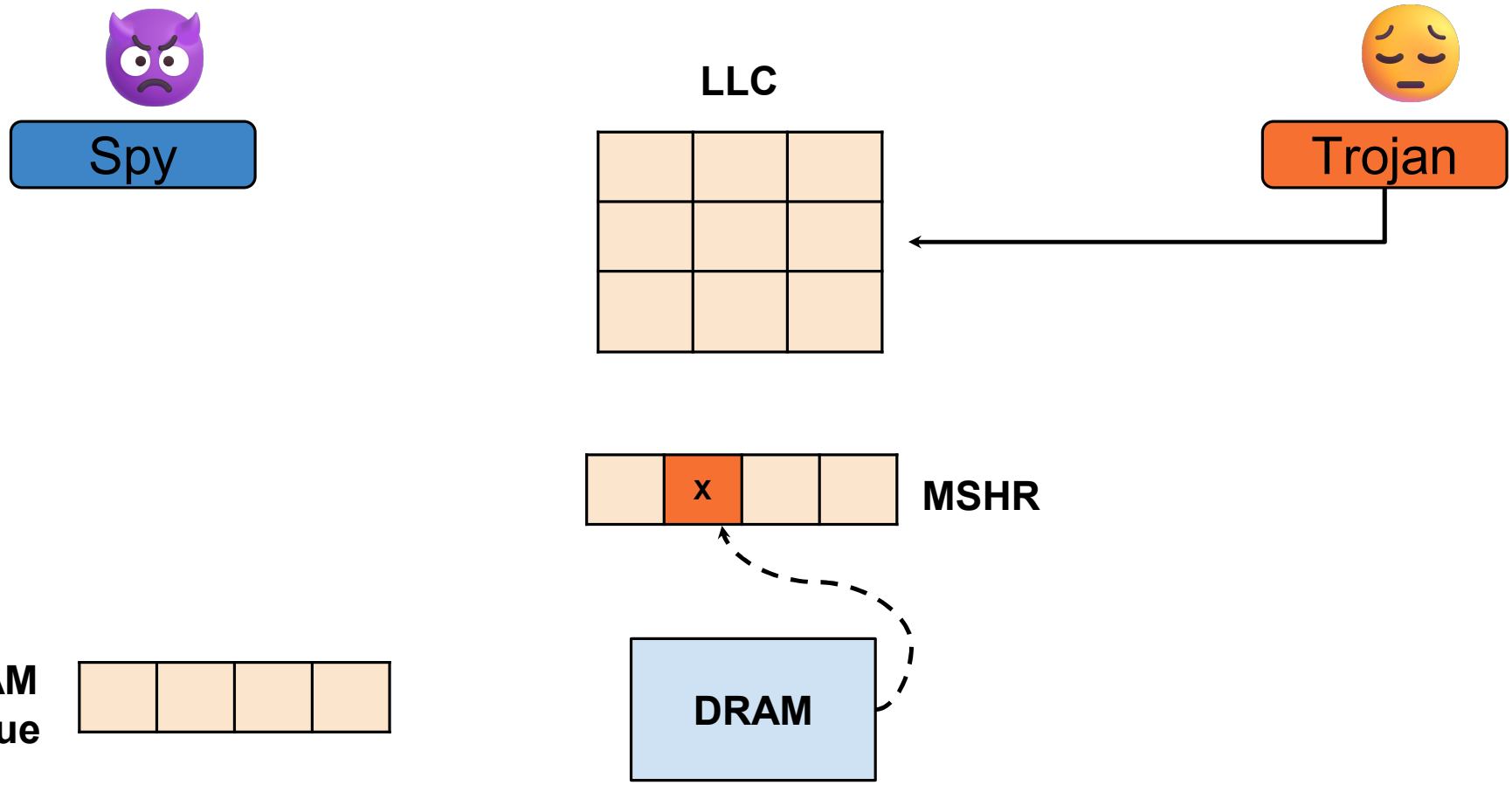
Working of TimeCache



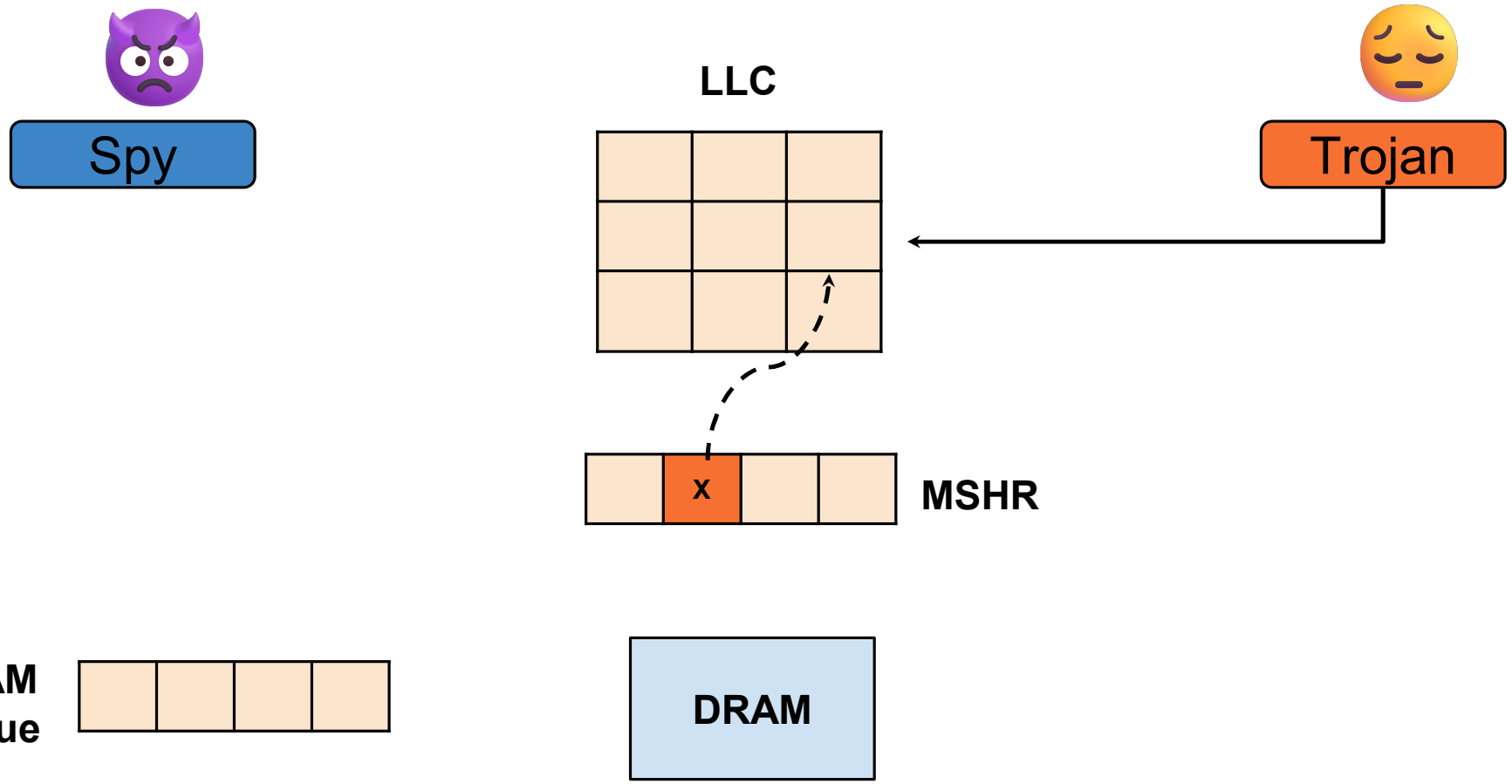
Working of TimeCache



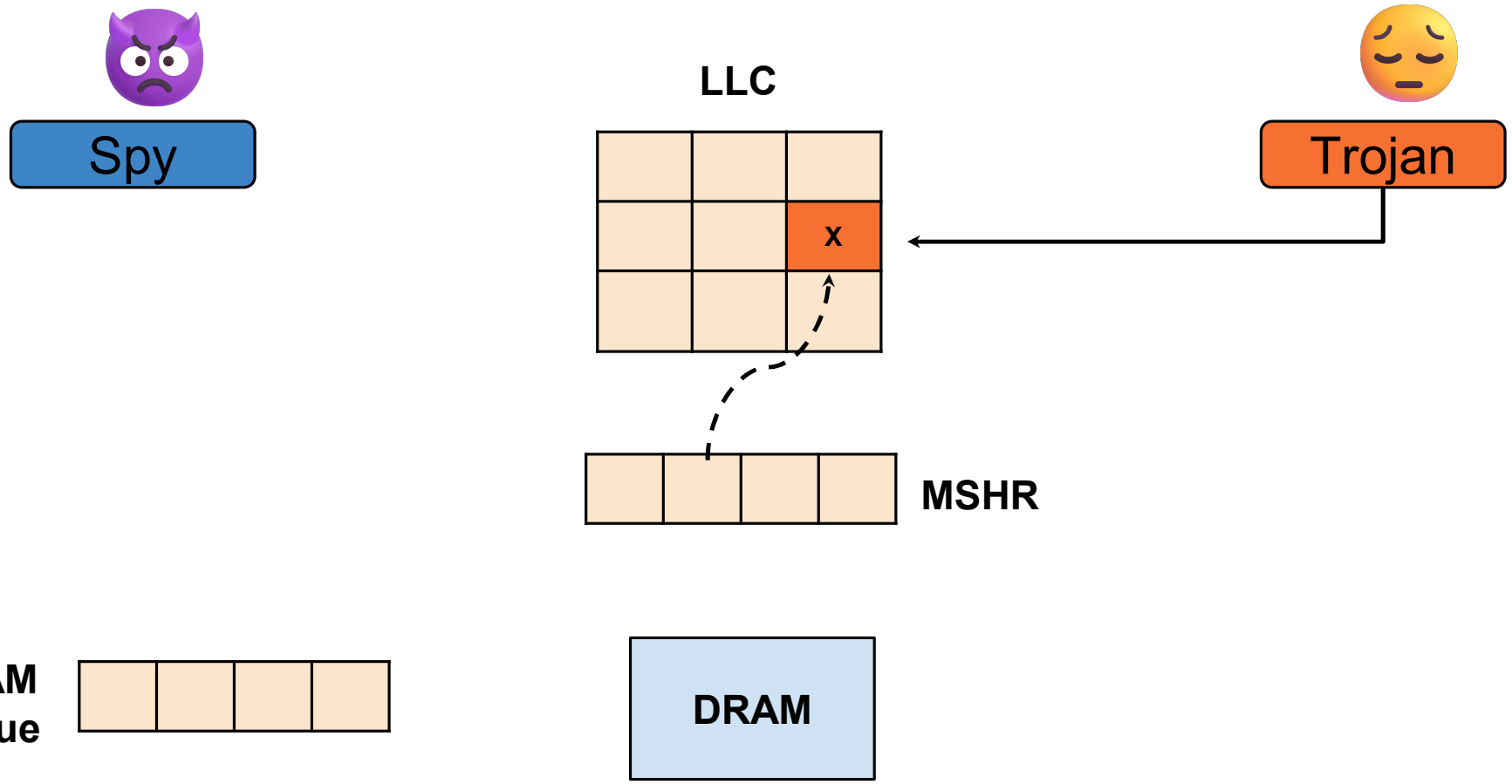
Working of TimeCache



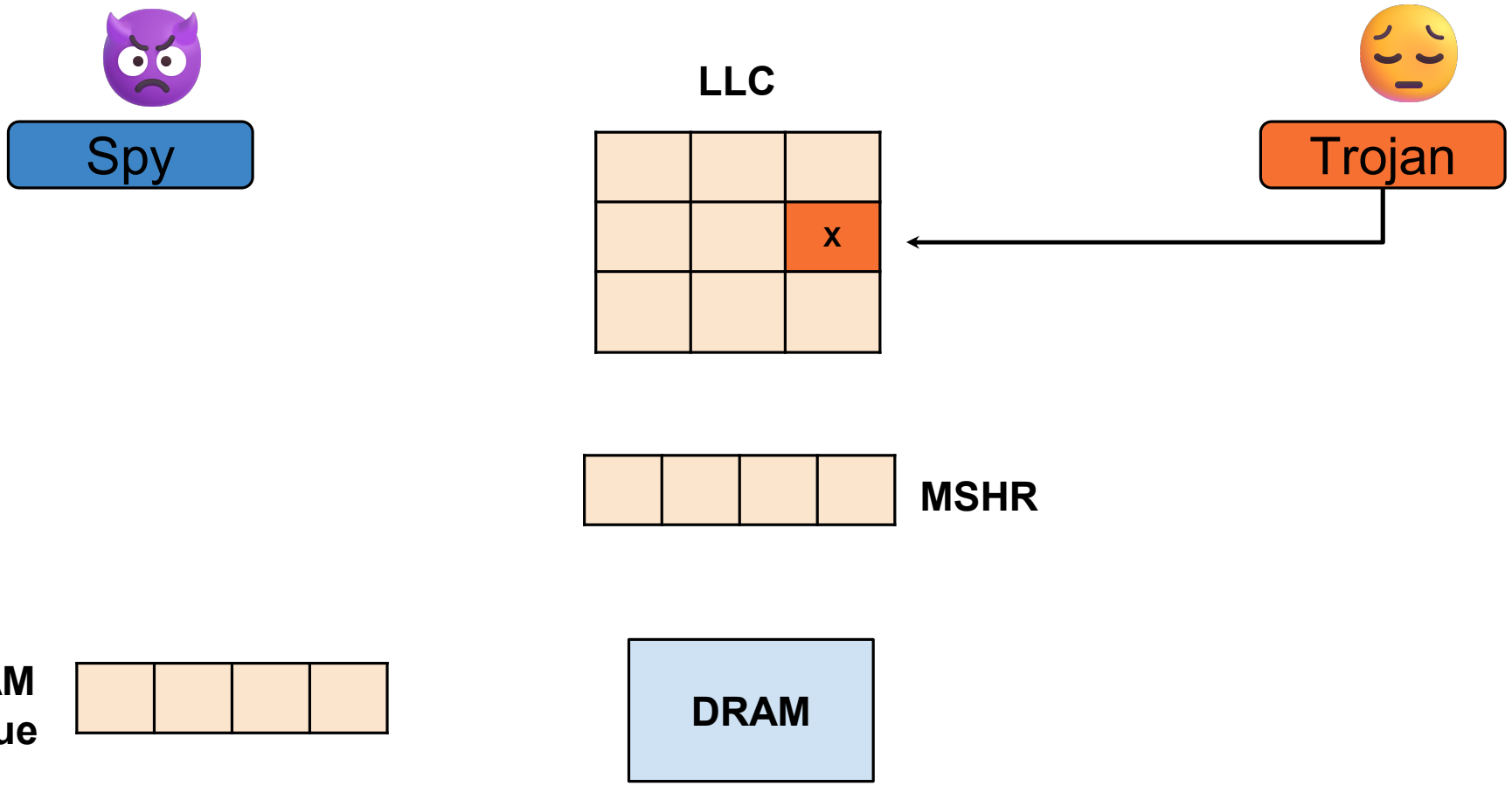
Working of TimeCache



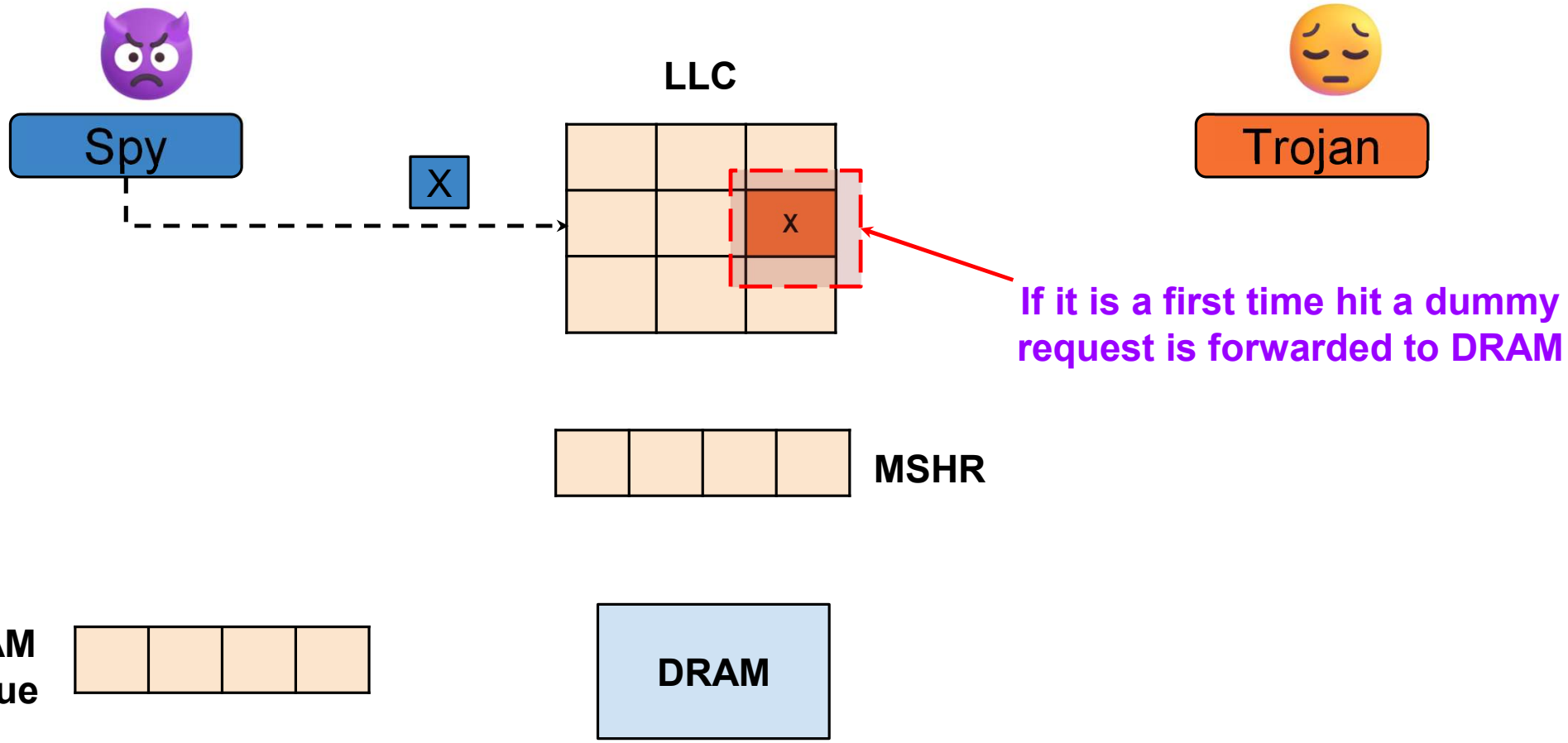
Working of TimeCache



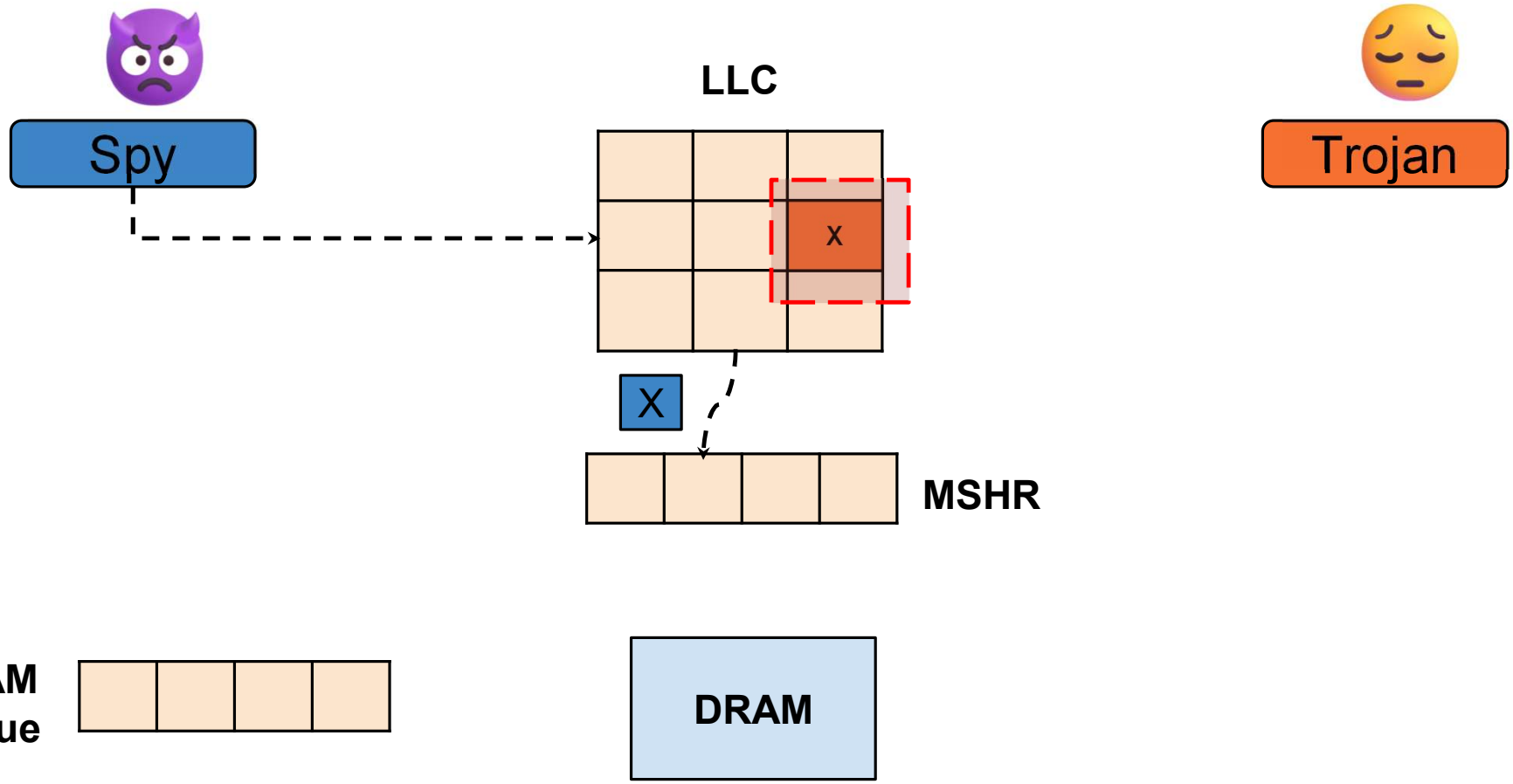
Working of TimeCache



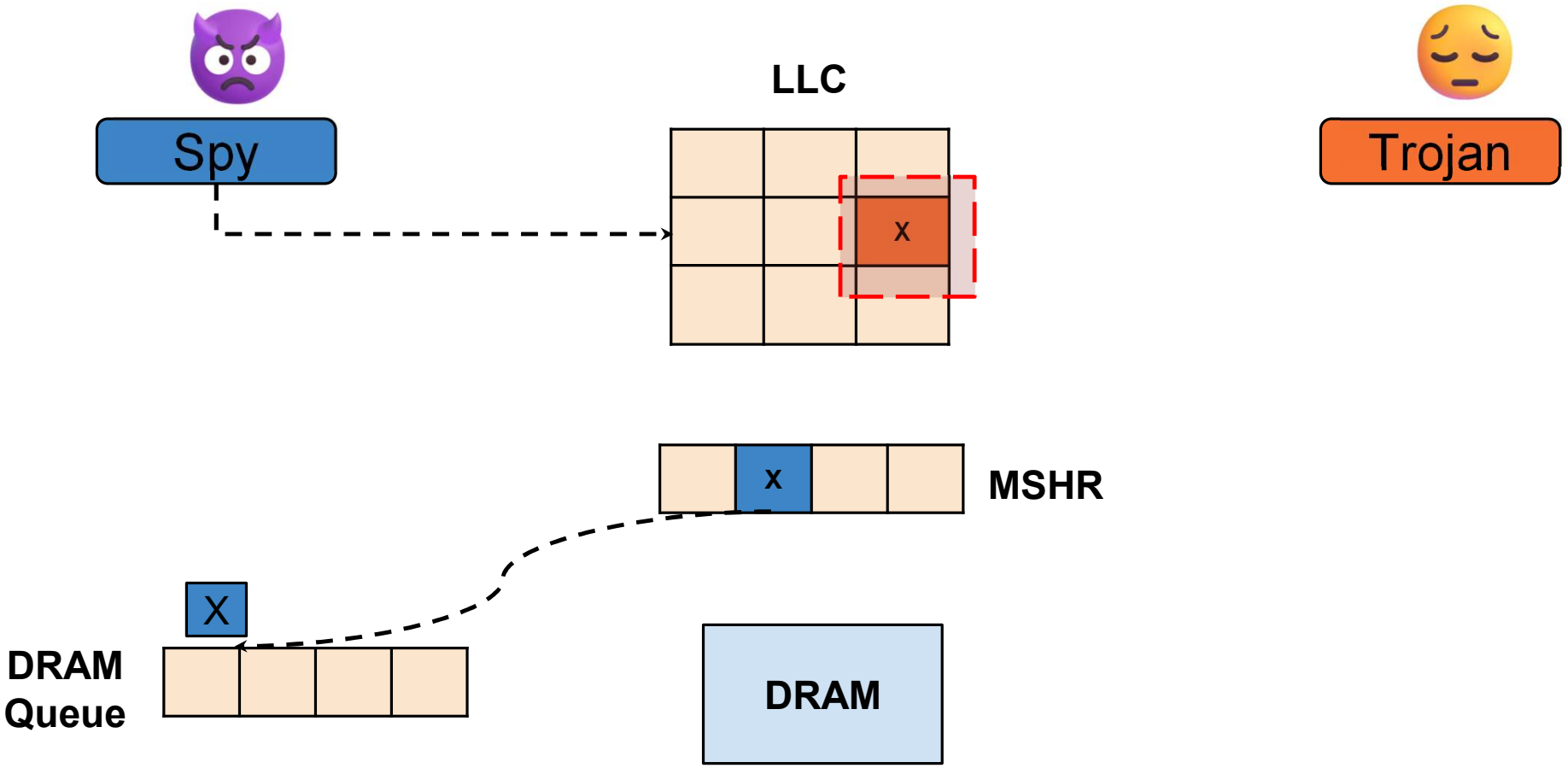
Working of TimeCache



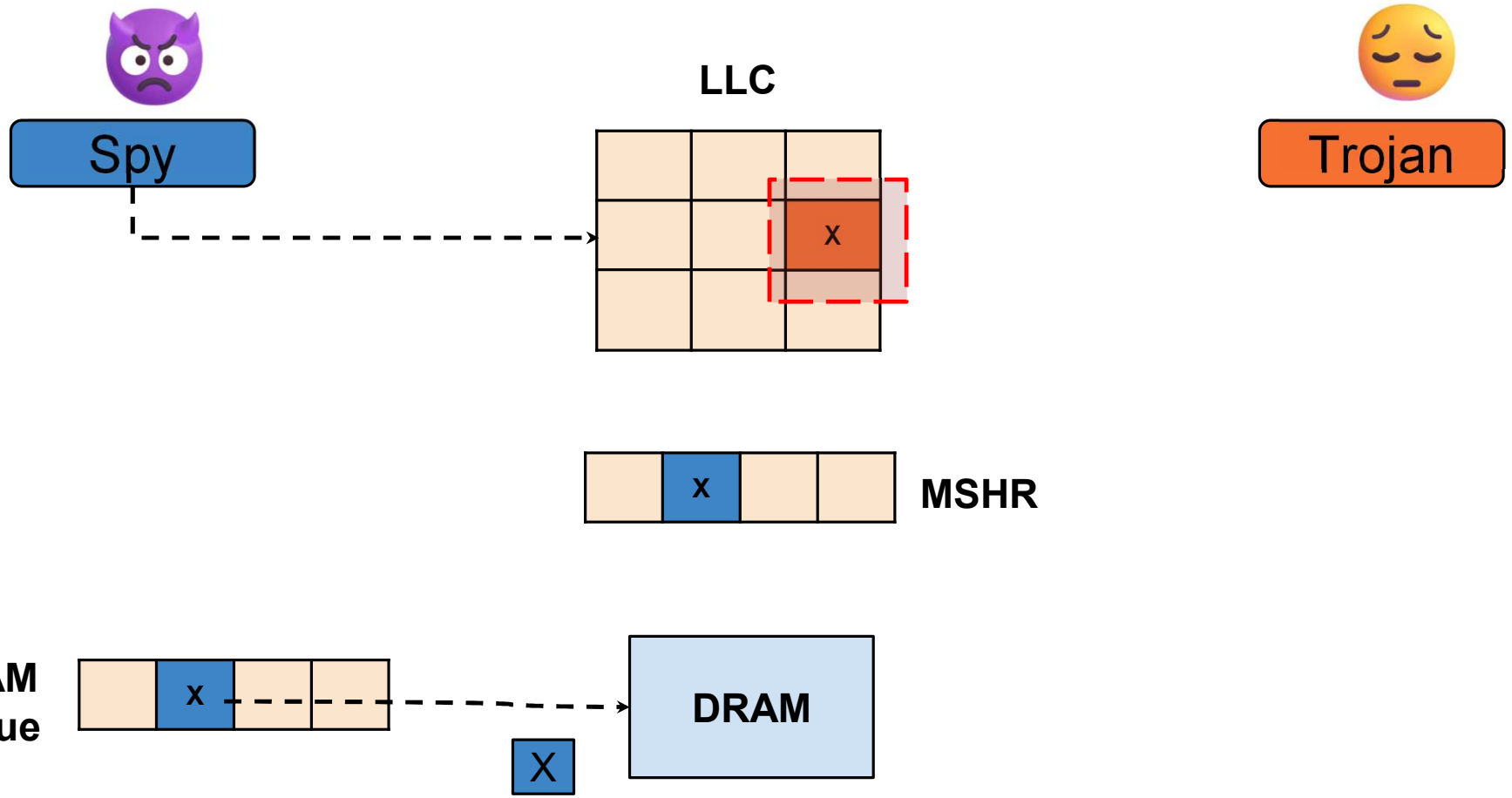
Working of TimeCache



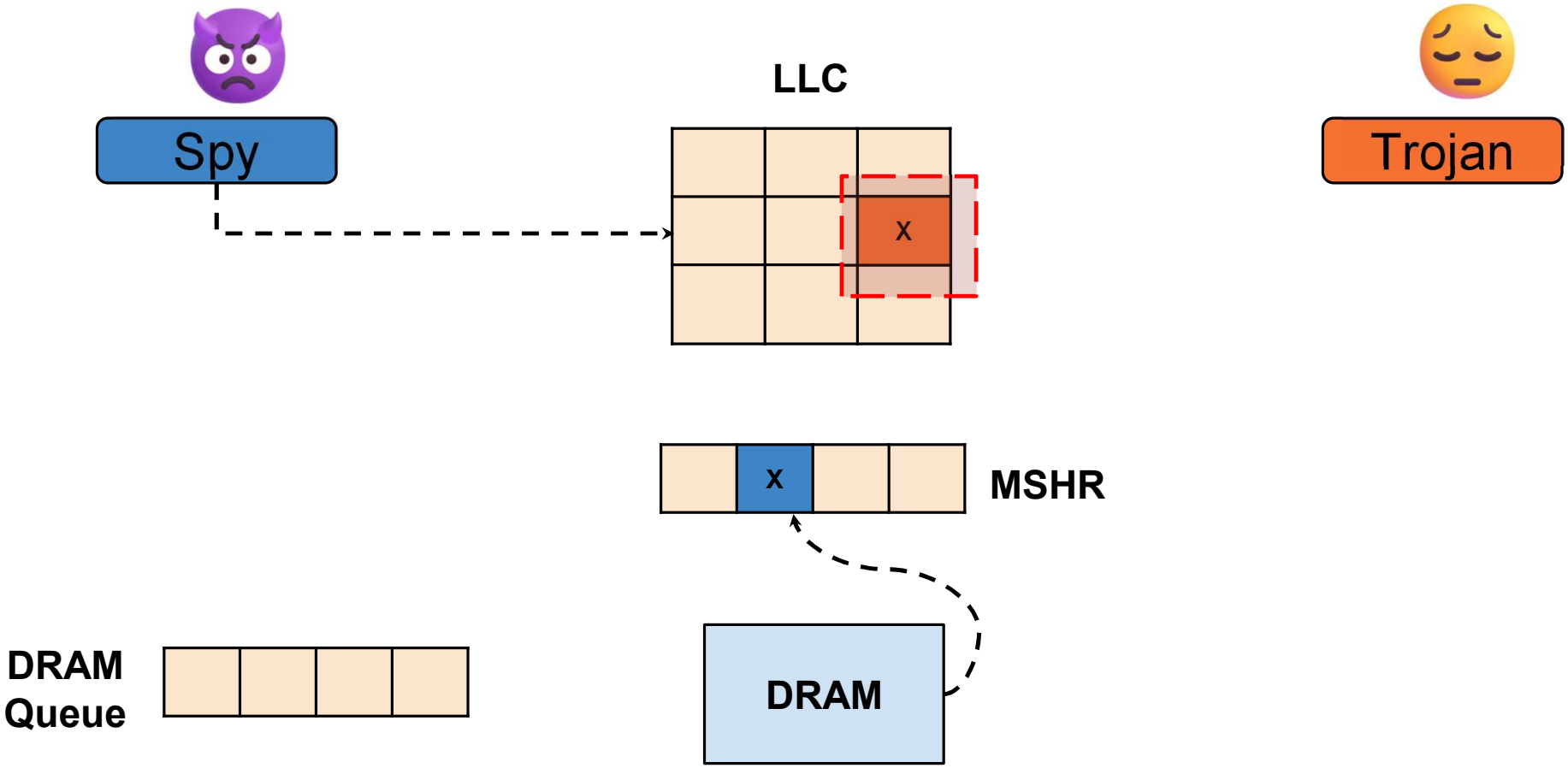
Working of TimeCache



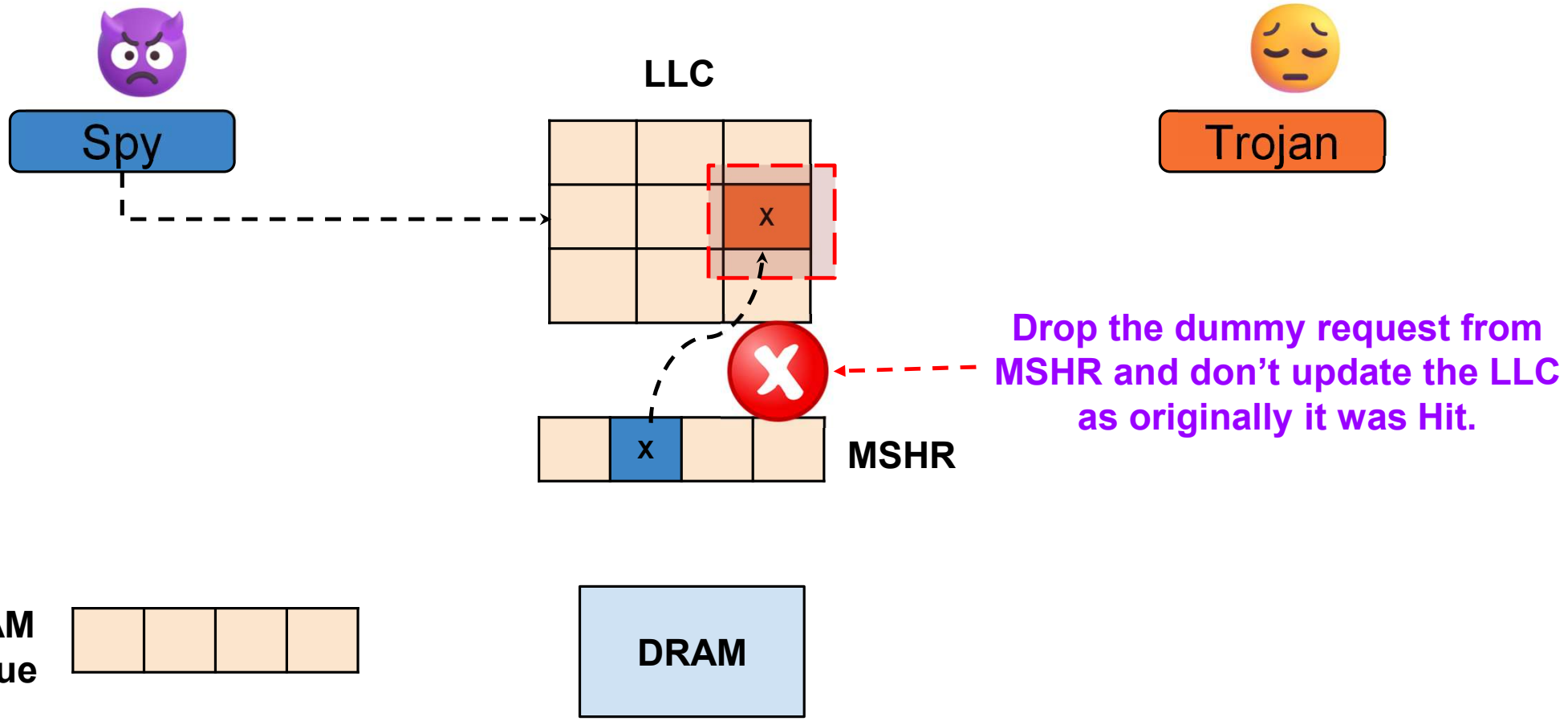
Working of TimeCache



Working of TimeCache



Working of TimeCache



Advanced Randomized Caches

Countermeasure for LLC Timing-Channel Attacks

Cross-core Attacks

Randomisation

Randomize cache placement and replacement to make predictions harder.

Partitioning

Partition the cache ways or sets among cores to limit information leakage.

Shared Memory/Other

Request Wait

Introduce random delays in memory requests to obscure timing information.

Data Duplication

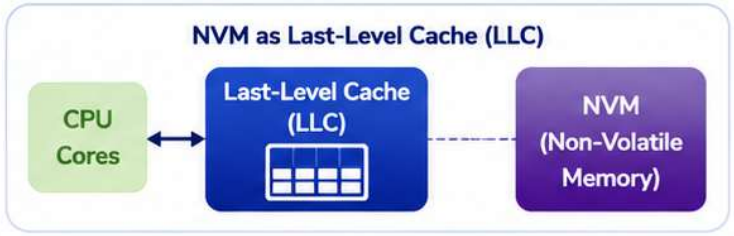
Duplicate data across multiple locations to eliminate contention-based leakage.

These techniques make cache behavior **less predictable**, reducing **information leakage** and **strengthening defense against** against LLC timing-channel attacks.

Issues with the existing Countermeasures

Non-Volatile Memory (NVM) as LLC

- **NVM** are used for designing both **Main Memories** and **Cache Memories**.
- They are either used Individually or combined with other technologies like **SRAM/DRAM** for **hybrid memories**.



Advantage of NVM:

- High density and lower static power consumption.
- Non-volatile and No periodic refresh like DRAM/eDRAM.

High Density | Low Static Power | No Refresh Required

Disadvantage of NVM:

- Limited write endurance and expensive write operation.

Limited Write Endurance | Expensive Write Operation

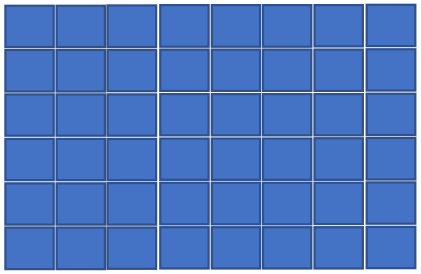
Solutions:

- Use in combination with **SRAM/eDRAM** to design hybrid cache and place write-intensive blocks into **SRAM**.
- Evenly distribute the write operations among all parts of the NVM memory. **Reduce the write variation.**

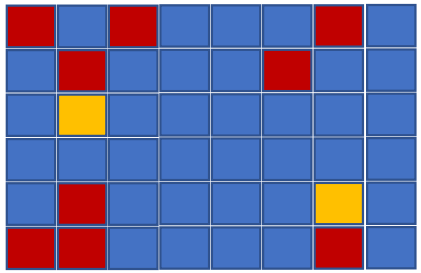
NVM + SRAM: Hybrid Cache (SRAM + NVM) | Place Write-Intensive Blocks into SRAM | Even Write Distribution → Reduce the Write Variation

NVM enables non-volatility and high density for last-level caches, but write limitations require intelligent management and hybrid solutions.

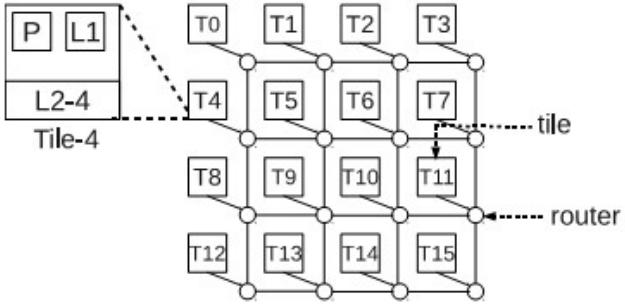
Handling Write Variation in NVM based LLCs



Set-associative cache



Intra-set and inter-set write variation



- **There are several write variations possible in modern NVM cache**
  - Intra bank variation
    - Intra-set variation
    - Inter-set variation.
  - Inter-bank variation.

- **Existing Ideas to resolve NVM issues:**

Endurance Enhancement:

- ***Uniform distribution of writes.***
- ***Example: [1]***

Improving the Performance:

- ***Efficient write bypassing at LLC [2, 3].***
- ***Multi-retention based LLC.***

1. S. Agarwal and H. K. Kapoor, “Improving the lifetime of non-volatile cache by write restriction,” IEEE Transactions on Computers, vol. 68, no. 9, pp. 1297–1312, 2019.
2. Prabuddha Sinha, Krishna Pratik Bv, **Shirshendu Das** and Venkata Kalyan Tavva, "PROLONG: Priority based Write Bypassing Technique for Longer Lifetime in STT-RAM based LLC", The International Symposium on Memory Systems (MEMSYS), 2024
3. K. Korgaonkar, I. Bhati, H. Liu, J. Gaur, S. Manipatruni, S. Subramoney, T. Karnik, S. Swanson, I. Young, and H. Wang, “Density Tradeoffs of Non-Volatile Memory as a Replacement for SRAM Based Last Level Cache,” in ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA), 2018, pp. 315–327

Advanced Randomized Caches

Countermeasure for LLC Timing-Channel Attacks

Cross-core Attacks

Randomisation

Randomize cache placement and replacement to make predictions harder.

Partitioning

Partition the cache ways or sets among cores to limit information leakage.

Shared Memory/Other

Request Wait

Introduce random delays in memory requests to obscure timing information.

Data Duplication

Duplicate data across multiple locations to eliminate contention-based leakage.

These techniques make cache behavior **less predictable**, reducing **information leakage** and **strengthening defense against** against LLC timing-channel attacks.

Issues with Randomized Countermeasures

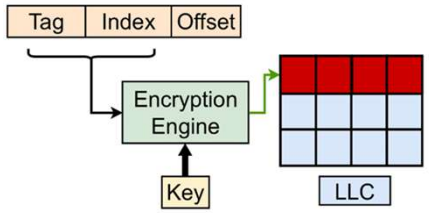
Issues with Remap-based Randomization Techniques [1,2,3]

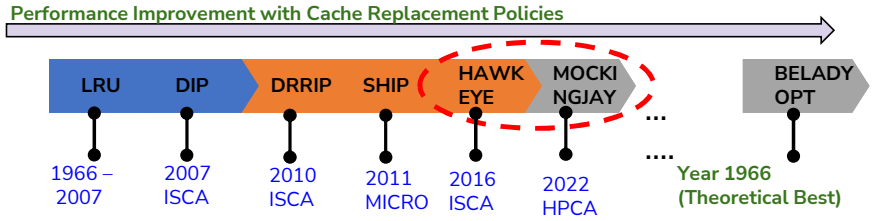
Issue 1: Performance issues with recent Replacement Policies.

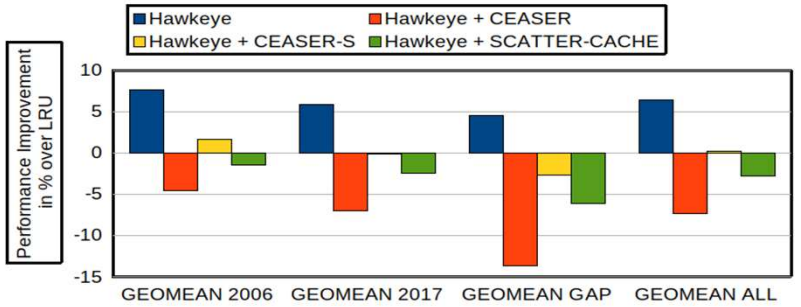
Issue 2: Performance degradation due to remapping.

Issue 3: Increased remap rate causes performance overhead and makes the techniques venerable to DoS attacks [4].

Issue 4: Cannot prevent shared-memory attack and occupancy attack [5,6].







1. M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," in *51st Annual International Symposium on Microarchitecture (MICRO)*, 2018.
2. M. K. Qureshi, "New attacks and defense for encrypted-address cache", *in Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19)*.
3. M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," *in 28th USENIX Security Symposium (USENIX Security 19)*, 2019.
4. Song, Wei, et al. "Randomizing Set-Associative Caches Against Conflict-Based Cache Side-Channel Attacks." in *IEEE Transactions on Computers* 73(4) 2024.
5. Ojha, Divya, and Sandhya Dwarkadas. "Timecache: Using time to eliminate cache side channels when sharing software." in *48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
6. Aditya S Gangwar, Prathamesh N Thanksale, **Shirshendu Das**, and Sudeepta Mishra, "Flush+earlyReload: Covert Channels Attack on Shared LLC Using MSHR Merging", in *DATE* 2024
7. Saileshwar, Gururaj, and Moinuddin Qureshi. "MIRAGE: Mitigating Conflict-Based cache attacks with a practical Fully-Associative design." in *30th USENIX Security Symposium (USENIX Security)*. 2021.
8. Bhatla, Anubhav, and Biswabandan Panda. "The Maya Cache: A Storage-efficient and Secure Fully-associative Last-level Cache." in *51st Annual International Symposium on Computer Architecture (ISCA)*. 2024.
9. Prathamesh Nitin Tanksale, Guru Raghava S. Seethiraju, **Shirshendu Das** and Venkata Kalyan Tavva, "RRR : Rethinking Randomized Remapping for High Performance Secured NVM LLC", *in IEEE HOST*, 2025.

Issues with Randomized Countermeasures

Issues with Remap-based Randomization Techniques [1,2,3]

Issue 1: Performance issues with recent Replacement Policies.

Issue 2: Performance degradation due to remapping.

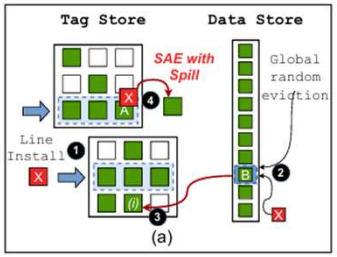
Issue 3: Increased remap rate causes performance overhead and makes the techniques venerable to DoS attacks [4].

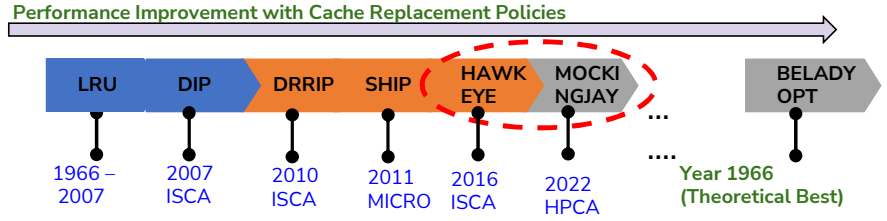
Issue 4: Cannot prevent shared-memory attack and occupancy attack [5,6].

Issues with Non-remap-based Randomization Techniques [7,8]

Issue 5: Only random replacement policy can be applied.

Issue 6: Cannot prevent shared-memory attack and occupancy attack.





1. M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," in *51st Annual International Symposium on Microarchitecture (MICRO)*, 2018.
2. M. K. Qureshi, "New attacks and defense for encrypted-address cache", *in Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19)*.
3. M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," *in 28th USENIX Security Symposium (USENIX Security 19)*, 2019.
4. Song, Wei, et al. "Randomizing Set-Associative Caches Against Conflict-Based Cache Side-Channel Attacks." in *IEEE Transactions on Computers* 73(4) 2024.
5. Ojha, Divya, and Sandhya Dwarkadas. "Timecache: Using time to eliminate cache side channels when sharing software." in *48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
6. Aditya S Gangwar, Prathamesh N Thanksale, **Shirshendu Das**, and Sudeepta Mishra, "Flush+earlyReload: Covert Channels Attack on Shared LLC Using MSHR Merging", in *DATE* 2024
7. Saileshwar, Gururaj, and Moinuddin Qureshi. "MIRAGE: Mitigating Conflict-Based cache attacks with a practical Fully-Associative design." in *30th USENIX Security Symposium (USENIX Security)*. 2021.
8. Bhatla, Anubhav, and Biswabandan Panda. "The Maya Cache: A Storage-efficient and Secure Fully-associative Last-level Cache." in *51st Annual International Symposium on Computer Architecture (ISCA)*. 2024.
9. Prathamesh Nitin Tanksale, Guru Raghava S. Seethiraju, **Shirshendu Das** and Venkata Kalyan Tavva, "RRR : Rethinking Randomized Remapping for High Performance Secured NVM LLC", *in IEEE HOST*, 2025.

Issues with Randomized Countermeasures

Issues with Remap-based Randomization Techniques [1,2,3]

Issue 1: Performance issues with recent Replacement Policies.

Issue 2: Performance degradation due to remapping.

Issue 3: Increased remap rate causes performance overhead and makes the techniques venerable to DoS attacks [4].

Issue 4: Cannot prevent shared-memory attack and occupancy attack [5,6].

Issues with Non-remap-based Randomization Techniques [7,8]

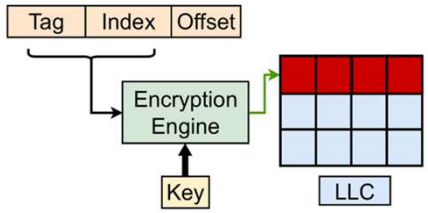
Issue 5: Only random replacement policy can be applied.

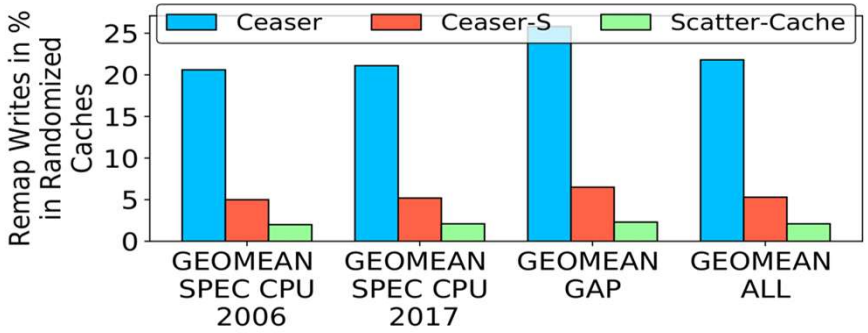
Issue 6: Cannot prevent shared-memory attack and occupancy attack.

Issues with Non-volatile memories [9]

Issue 7: High write latency creates issues with Remap-based Randomization Techniques.

Issue 8: Bypassing writes is not safe in both randomization techniques.





1. M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," in *51st Annual International Symposium on Microarchitecture (MICRO)*, 2018.
2. M. K. Qureshi, "New attacks and defense for encrypted-address cache", *in Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19)*.
3. M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," *in 28th USENIX Security Symposium (USENIX Security 19)*, 2019.
4. Song, Wei, et al. "Randomizing Set-Associative Caches Against Conflict-Based Cache Side-Channel Attacks." in *IEEE Transactions on Computers* 73(4) 2024.
5. Ojha, Divya, and Sandhya Dwarkadas. "Timecache: Using time to eliminate cache side channels when sharing software." in *48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
6. Aditya S Gangwar, Prathamesh N Thanksale, **Shirshendu Das**, and Sudeepta Mishra, "Flush+earlyReload: Covert Channels Attack on Shared LLC Using MSHR Merging", in *DATE* 2024
7. Saileshwar, Gururaj, and Moinuddin Qureshi. "MIRAGE: Mitigating Conflict-Based cache attacks with a practical Fully-Associative design." in *30th USENIX Security Symposium (USENIX Security)*. 2021.
8. Bhatla, Anubhav, and Biswabandan Panda. "The Maya Cache: A Storage-efficient and Secure Fully-associative Last-level Cache." in *51st Annual International Symposium on Computer Architecture (ISCA)*. 2024.
9. Prathamesh Nitin Tanksale, Guru Raghava S. Seethiraju, **Shirshendu Das** and Venkata Kalyan Tavva, "RRR : Rethinking Randomized Remapping for High Performance Secured NVM LLC", *in IEEE HOST*, 2025.

Our Plans to Resolve these Issues

Issues with Remap-based Randomization Techniques [1,2,3]

Issue 1: Performance issues with recent Replacement Policies.

Issue 2: Performance degradation due to remapping.

Issue 3: Increased remap rate causes performance overhead and makes the techniques venerable to DoS attacks [4].

Issue 4: Cannot prevent shared-memory attack and occupancy attack [5,6].

Issues with Non-remap-based Randomization Techniques [7,8]

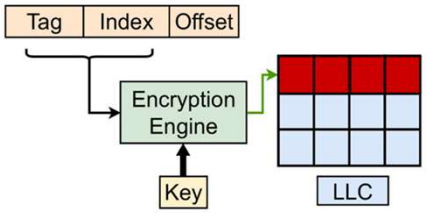
Issue 5: Only random replacement policy can be applied.

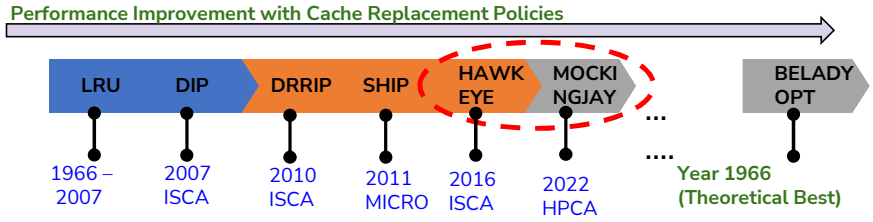
Issue 6: Cannot prevent shared-memory attack and occupancy attack.

Issues with Non-volatile memories [9]

Issue 7: High write latency creates issues with Remap-based Randomization Techniques.

Issue 8: Bypassing writes is not safe in both randomization techniques.



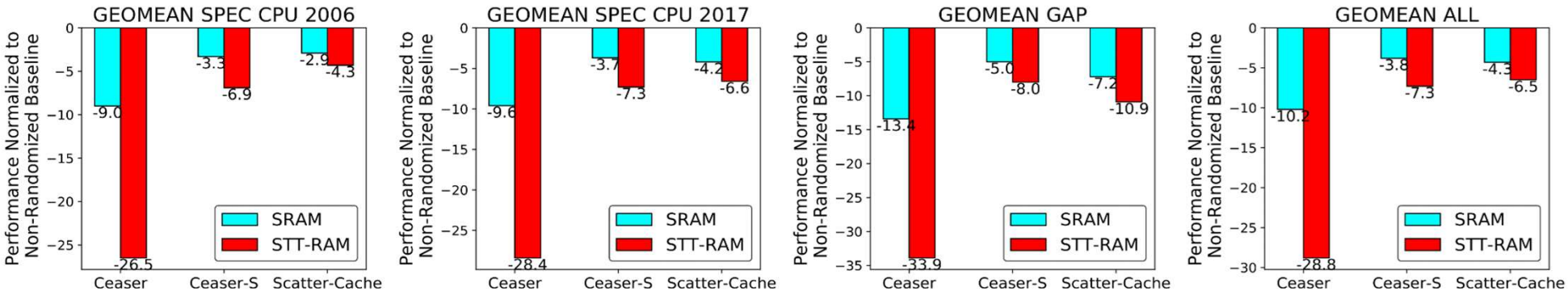


RR: Rethinking Randomization for Secure and Efficient LLCs

RRR: Rethinking Randomized Remapping for High Performance Secured NVM LLC

IEEE International Symposium on Hardware Oriented Security and Trust (HOST), 2025

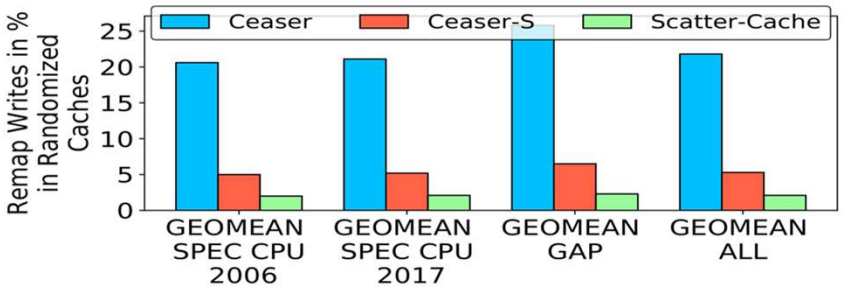
RRR: Rethinking Randomized Remapping



Average performance of a 4-core system with randomized LLC across 97 workloads from SPEC CPU 2006/2017, and GAP benchmarks. The randomized LLC of SRAM and STT-RAM are compared to their respective non-randomized baseline configurations.

Randomized Cache	SRAM (Normalized to non-randomized SRAM)	STT-RAM (Normalized to non-randomized STT-RAM)
Ceaser	- 10.2 %	- 28.8 %
Ceaser-S	- 3.8 %	- 7.3 %
Scatter-Cache	- 4.3 %	- 6.5 %

Geomean All comparison of above figures

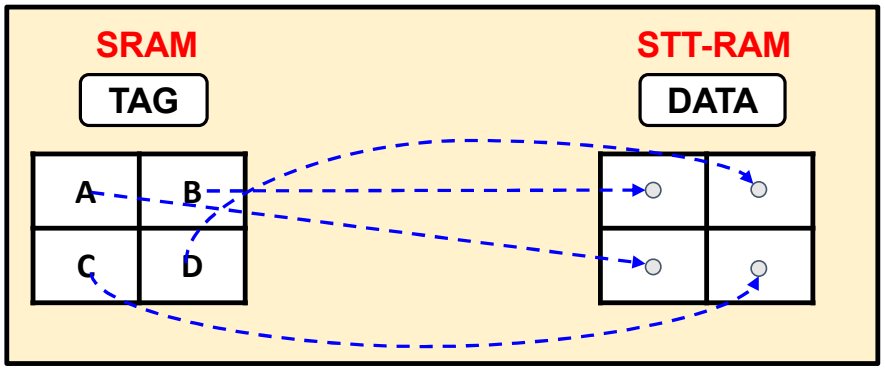


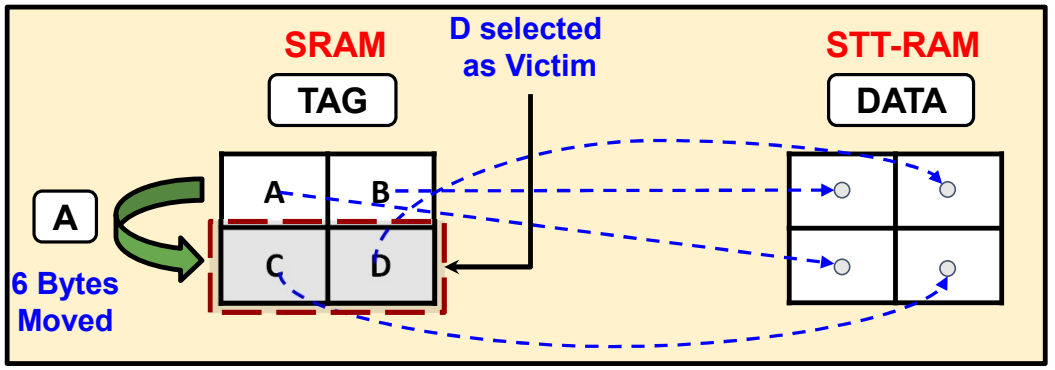
Percentage of remap writes in SPEC CPU 2006/2017, and GAP workloads.

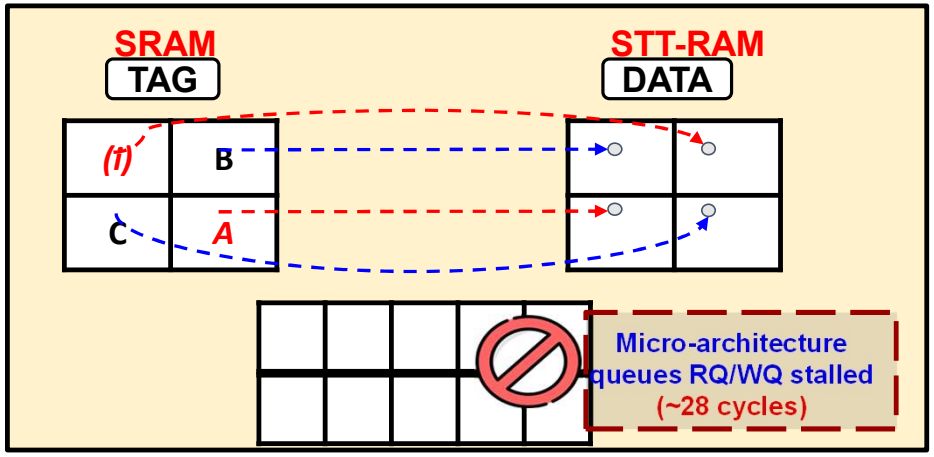
Remapping stalls the microarchitecture queues RQ/WQ and Remap Writes contributes ~20%, ~5%, and ~2% of total writes in Ceaser, Ceaser-S, and Scatter-cache respectively

Securing the STT-RAM LLC is more expensive *(in terms of performance)* than securing the traditional SRAM LLC.

RRR: The Proposed Idea







1. Qureshi, Moinuddin K., David Thompson, and Yale N. Patt. "The V-Way cache: demand-based associativity via global replacement." *in 32nd International Symposium on Computer Architecture (ISCA)*, 2005.
2. Bhatla, Anubhav, and Biswabandan Panda. "The Maya Cache: A Storage-efficient and Secure Fully-associative Last-level Cache." in *51st Annual International Symposium on Computer Architecture (ISCA)*. 2024.

RRR: Rethinking Randomized Remapping

Approach	Tag array Accesses Time Taken	Data array Accesses Time Taken	Total Time
Traditional Remap (SRAM)	1 R + 1 W (12 cycles)	1 R + 1 W (28 cycles)	28/40 cycles (Parallel/Serial) access
Traditional Remap (STT-RAM)	1 R + 1 W (12 cycles)	1 R + 1 W (84 cycles)	84/96 cycles (Parallel/Serial) access
RRR	2 R + 2 W (28 cycles)	— —	28 cycles

Cycles required for performing writes in Randomized Cache with different memory technologies.

Randomized Cache	Traditional Remap (Normalized to non-randomized STT-RAM)	RRR Remap (Normalized to non-randomized STT-RAM)
Ceaser	- 28.8 %	- 11 %
Ceaser-S	- 7.3 %	- 3.9 %
Scatter-Cache	- 6.5 %	- 5.5 %

Performance recovery of RRR as compared to traditional Randomized Cache

RRR saves ~66 % of remap time per remap block

RRR saves ~94 % of dynamic energy per remap block

Our Plans to Resolve these Issues

Issues with Remap-based Randomization Techniques [1,2,3]

Issue 1: Performance issues with recent Replacement Policies.

Issue 2: Performance degradation due to remapping.

Issue 3: Increased remap rate causes performance overhead and makes the techniques venerable to DoS attacks [4].

Issue 4: Cannot prevent shared-memory attack and occupancy attack [5,6].

Issues with Non-remap-based Randomization Techniques [4,5]

Issue 5: Only random replacement policy can be applied.

Issue 6: Cannot prevent shared-memory attack and occupancy attack.

Issues with Non-volatile memories [9]

Issue 7: High write latency creates issues with Remap-based Randomization Techniques.

Issue 8: Bypassing writes is not safe in both randomization techniques.

RR: Rethinking Randomization for Secure and Efficient LLCs

RRR: Rethinking Randomized Remapping for High Performance Secured NVM LLC

IEEE International Symposium on Hardware Oriented Security and Trust (HOST), 2025

Advanced Randomized Caches

Countermeasure for LLC Timing-Channel Attacks

Cross-core Attacks

Randomisation

Randomize cache placement and replacement to make predictions harder.

Partitioning

Partition the cache ways or sets among cores to limit information leakage.

Shared Memory/Other

Request Wait

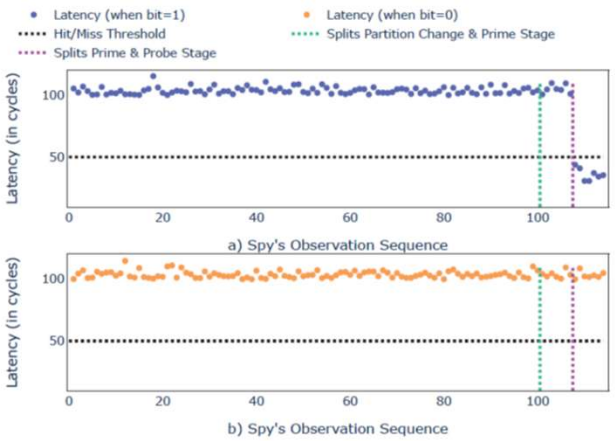
Introduce random delays in memory requests to obscure timing information.

Data Duplication

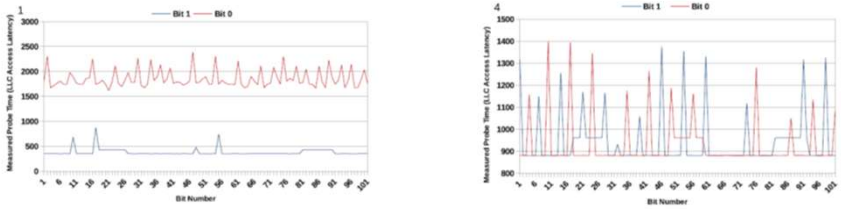
Duplicate data across multiple locations to eliminate contention-based leakage.

These techniques make cache behavior **less predictable**, reducing **information leakage** and **strengthening defense against** against LLC timing-channel attacks.

Dynamic LLC Partition and the Proposed CCA Attack

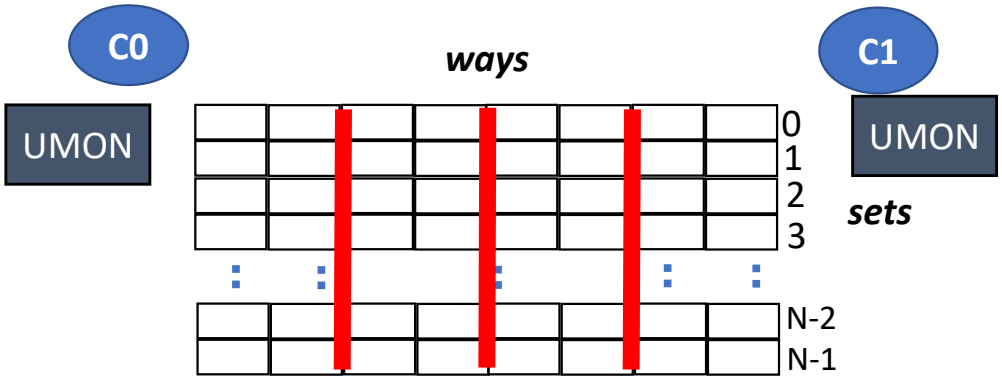


CCA on Dynamic Cache Partitioning [1]

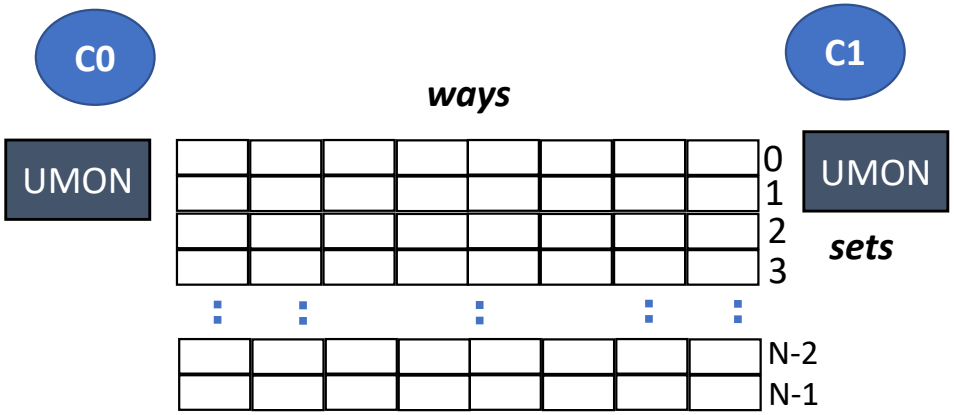


CCA mitigation with Restricted Cache Partitioning [2]

Cache Partitioning Example



CCA Attack on Dynamic Partitioning



1. Anurag Agarwal, Jaspinder Kaur, and **Shirshendu Das**, "Exploiting Secrets by Leveraging Dynamic Cache Partitioning of Last Level Cache," Design, Automation and Test in Europe Conference (DATE), 2021.
2. Jaspinder Kaur, and **Shirshendu Das**, "ACPC: Covert Channel Attack on Last Level Cache using Dynamic Cache Partitioning," 24th International Symposium on Quality Electronic Design (ISQED),, 2023.
3. N. R. Holtryd, M. Manivannan and P. Stenström, "SCALE: Secure and Scalable Cache Partitioning," *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2023, pp. 68-79

Targeted Partitioning

The most recent and efficient replacement policies show **significant degradation** while applied on **Randomized** cache.

Attack Scenario

- Spy (C1) and Trojan (C2) are co-located on the same last-level cache (LLC).
- Spy monitors cache behavior to infer Trojan's presence and activity.

Defense Goal

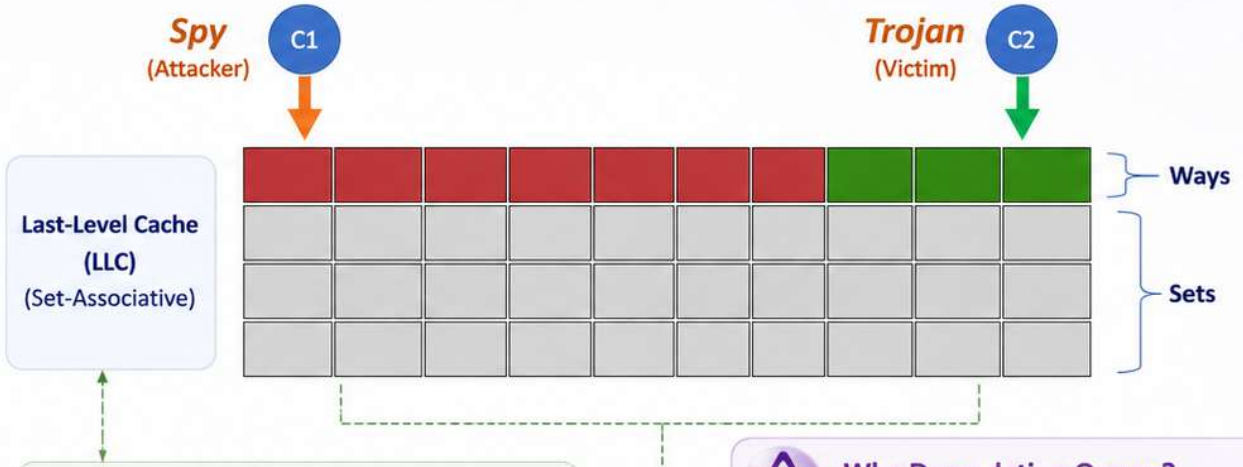
- Isolate cache ways to limit information leakage between applications.
- Detect abnormal interference using an Attack Detector.

Attack Detector

- Monitors cache events (misses, evictions, access patterns).
- Identifies abnormal degradation caused by adversarial interference.

Key Insight

- Naïve or recent replacement policies perform poorly under **randomized cache**.
- Targeted partitioning + detection mitigate cross-core covert channel attacks.



Attack Detector

Monitors cache behavior and detects performance degradation caused by adversarial interference.

Why Degradation Occurs?

- Randomized cache + modern policies reduce prediction accuracy.
- Increased conflict misses and evictions.
- Leakage of timing and access pattern information.

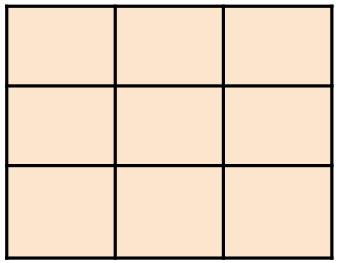
Targeted partitioning combined with attack detection effectively reduces leakage and restores performance under randomized cache.

1. Jaspinder Kaur and **Shirshendu Das**, "TPPD: Targeted Pseudo Partitioning based Defence for cross-core covert channel attacks," Journal of Systems Architecture, vol. 135, p. 102805, 2023.
2. Jaspinder Kaur and **Shirshendu Das**, "RSPP: Restricted Static Pseudo-Partitioning for Mitigation of Cross-Core Covert Channel Attacks," ACM Transactions on Design Automation of Electronic Systems (TODAES), Accepted, Nov 2023.

Can we exploit TimeCache?

Spy

LLC



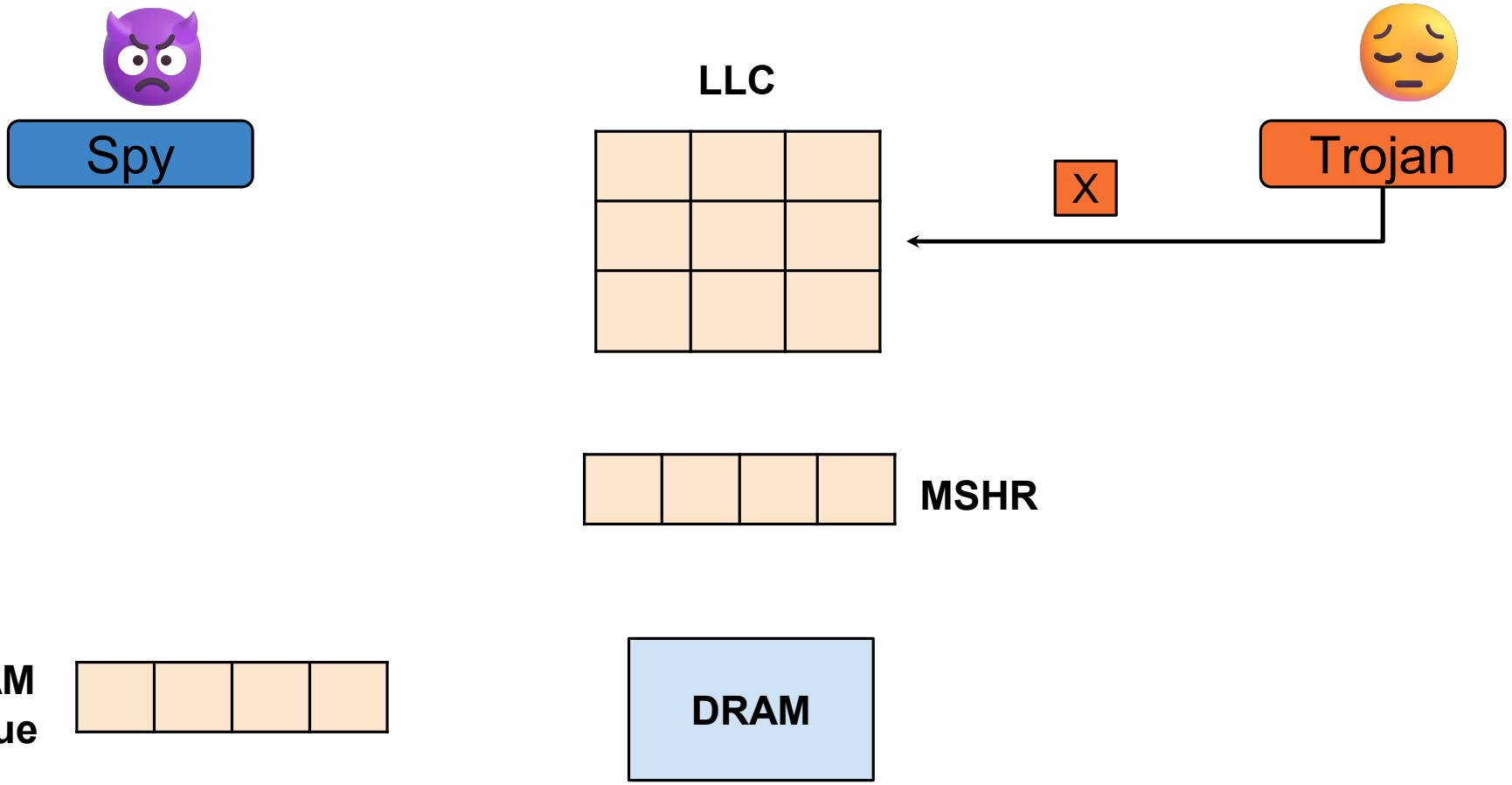
Trojan

MSHR

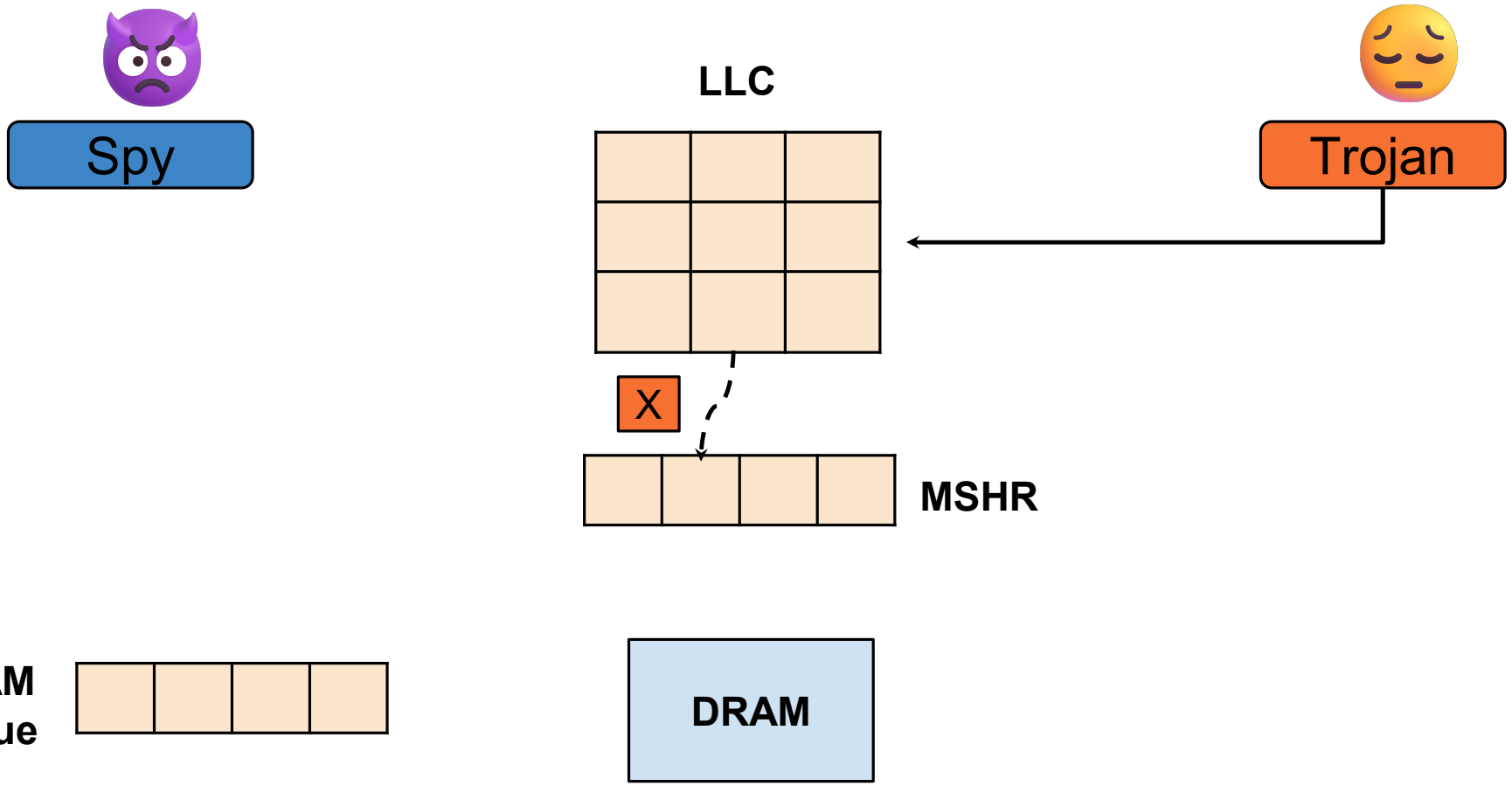
DRAM Queue

DRAM

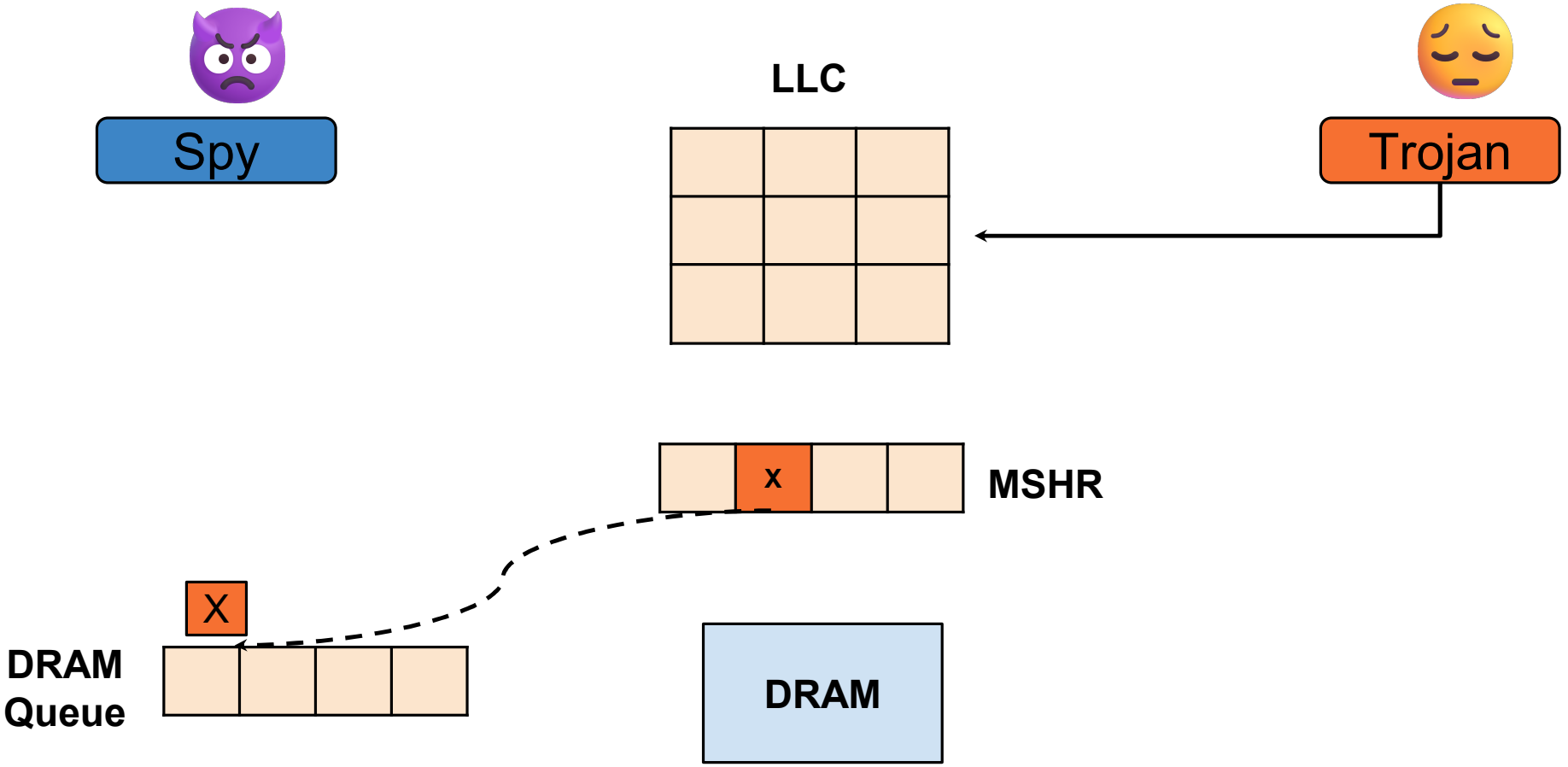
Can we exploit TimeCache?



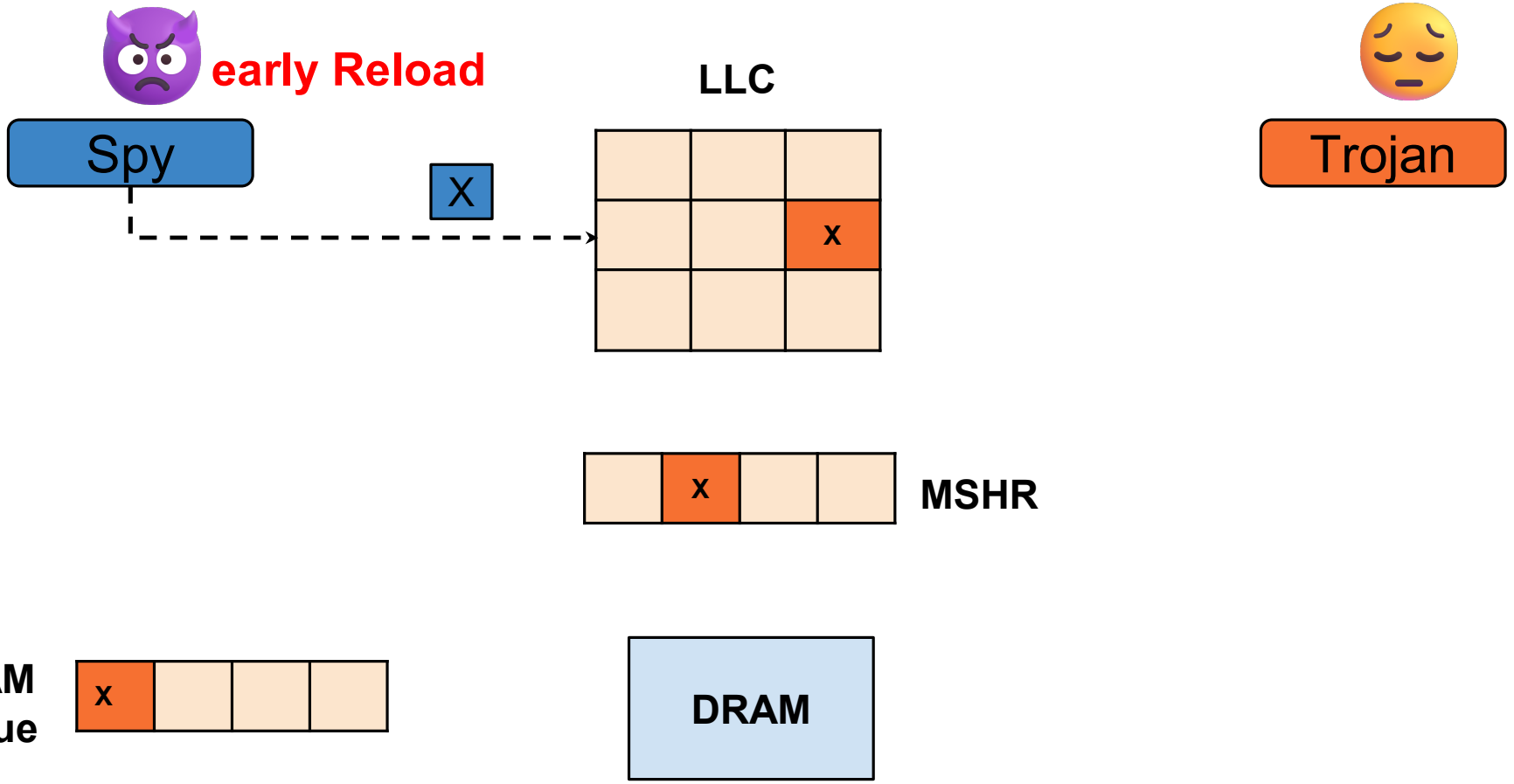
Can we exploit TimeCache?



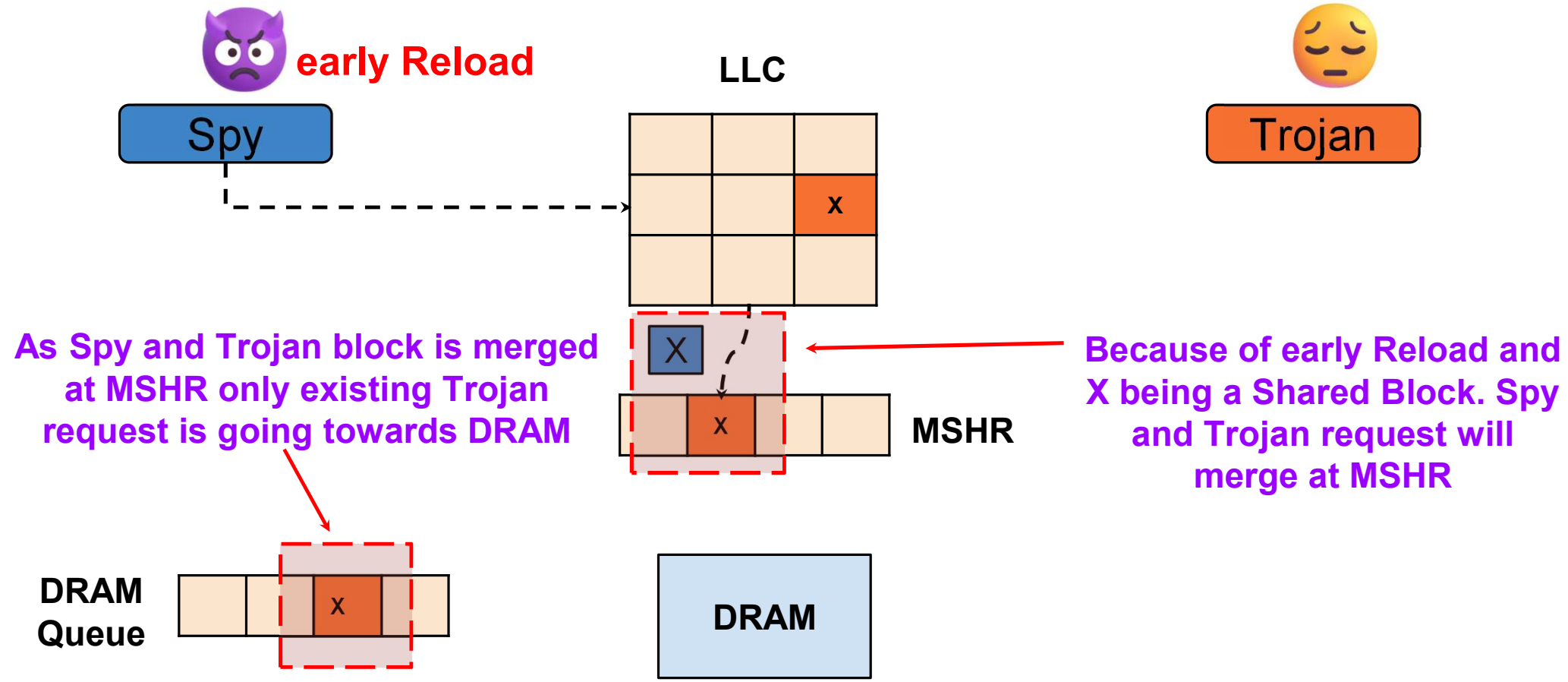
Can we exploit TimeCache?



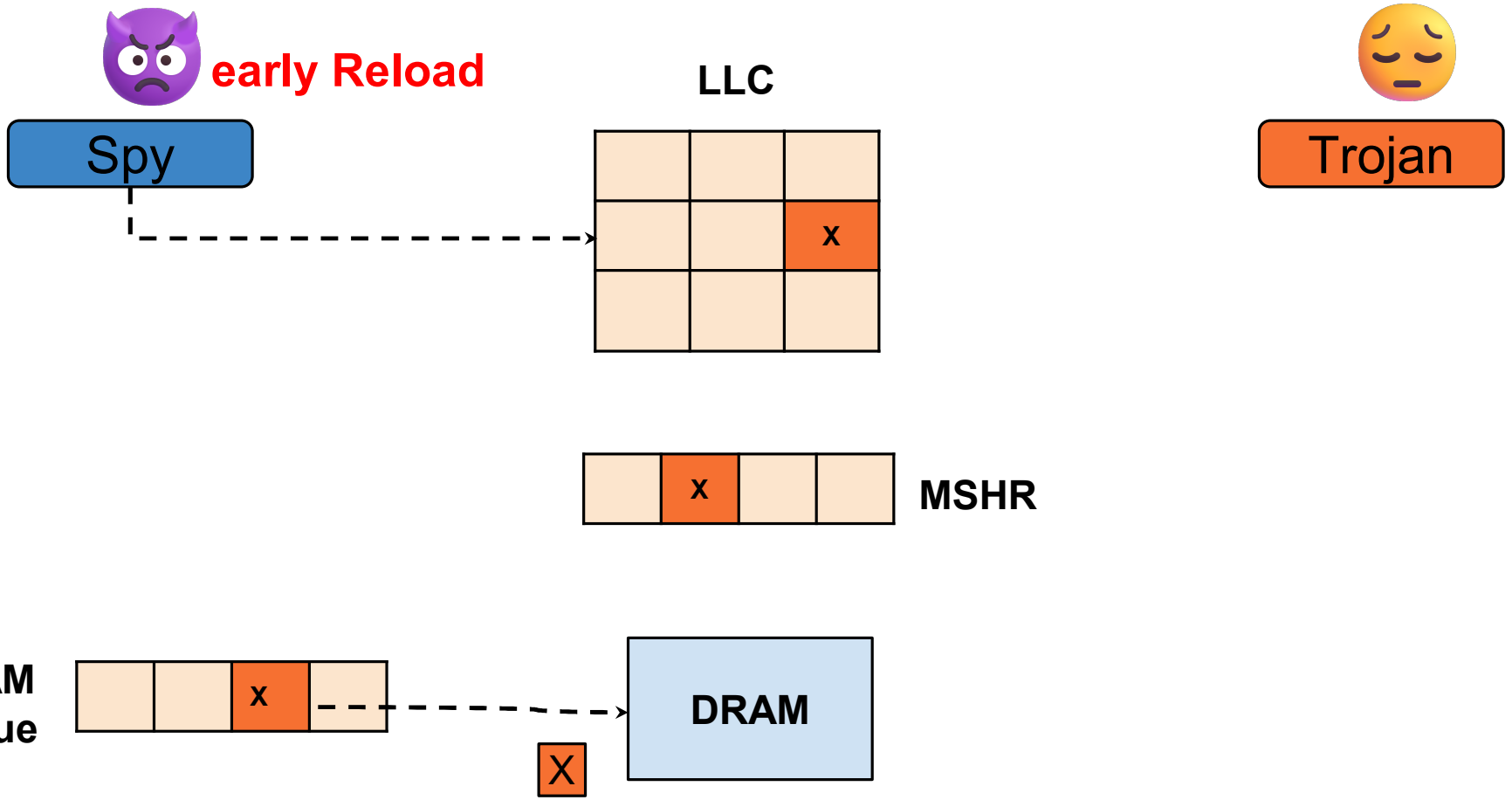
Can we exploit TimeCache?



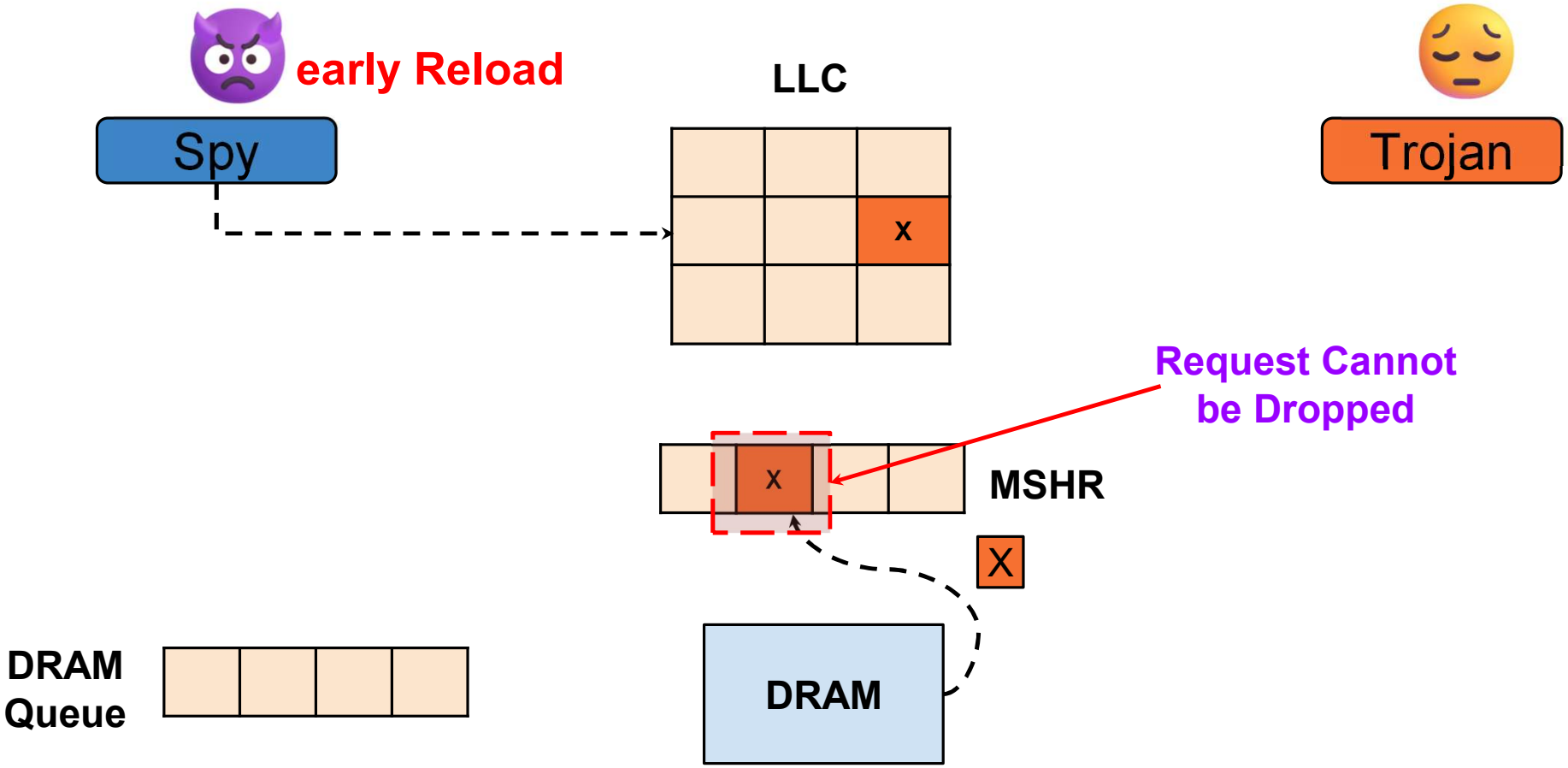
Can we exploit TimeCache?



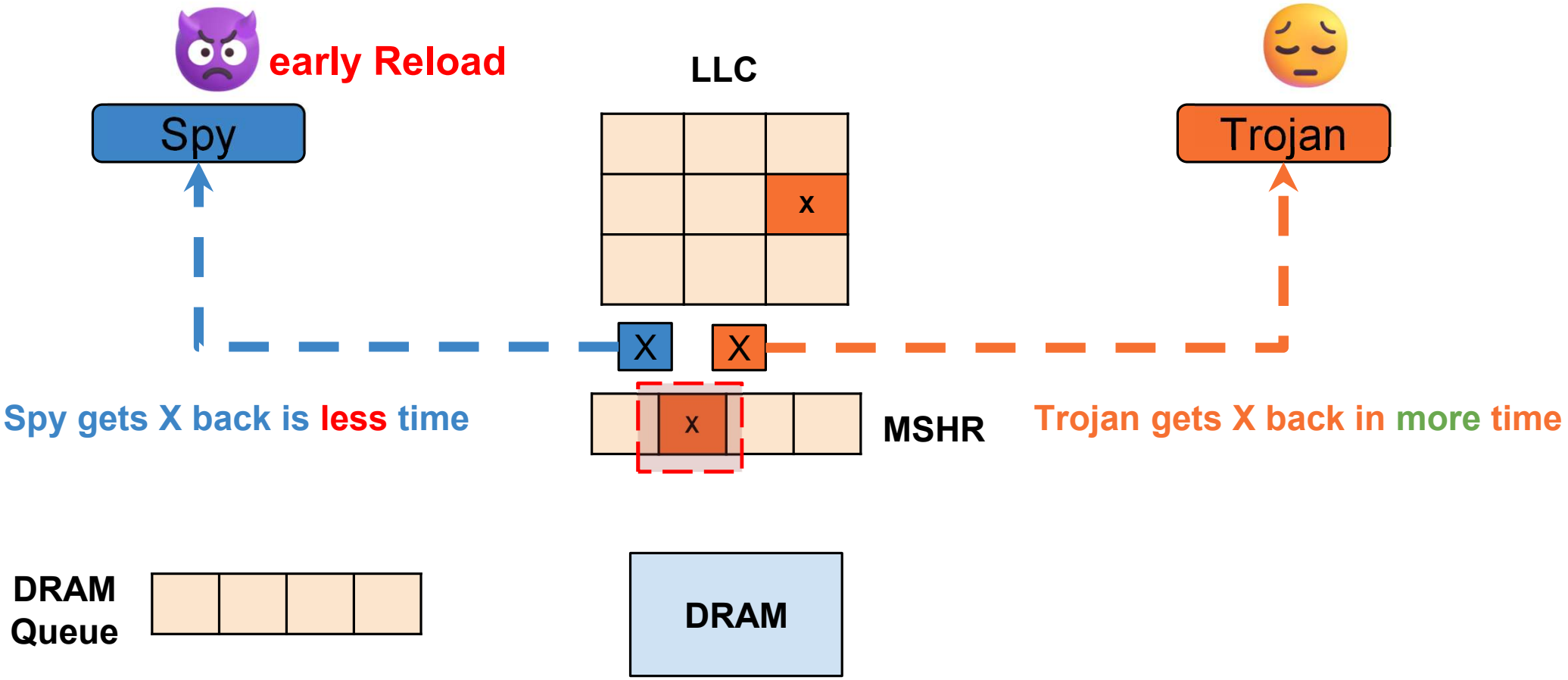
Can we exploit TimeCache?



Can we exploit TimeCache?



Can we exploit TimeCache?



Confirmed Merge with early reload?

- For a confirmed MSHR merge, the spy should issue the reload between

$$T_{hit}+p \text{ and } T_{hit}+ (T_{mm} -(T_c+T_q))$$

- p is padding delay, and p < (Tc+Tq).
- Any time before and after may miss the MSHR merging.
- Observed 100% accurate merging with p between 0.5Tmm & 0.8Tmm.

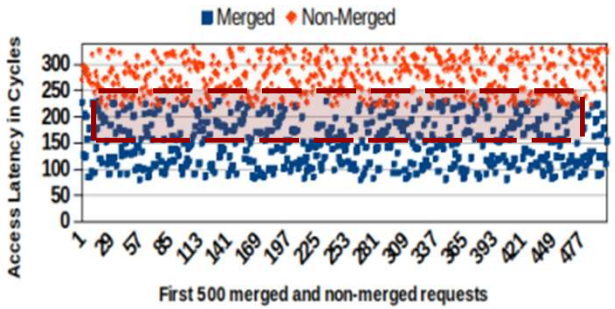
where, T_{Hit} is The minimum time required to access a block if hit in LLC ,

T_{mm} is the minimum time required to get the block from main memory after the request is issued from the LLC

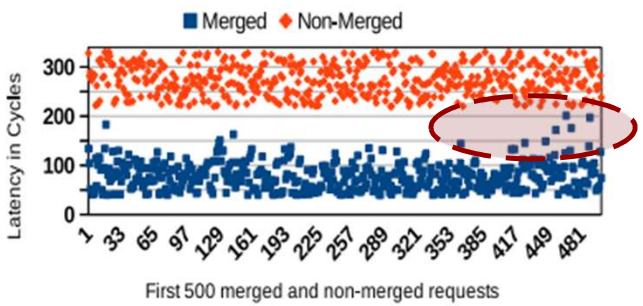
T_c is the time required to communicate a request from the core to the LLC

T_q is the average waiting time in the request queue for LLC

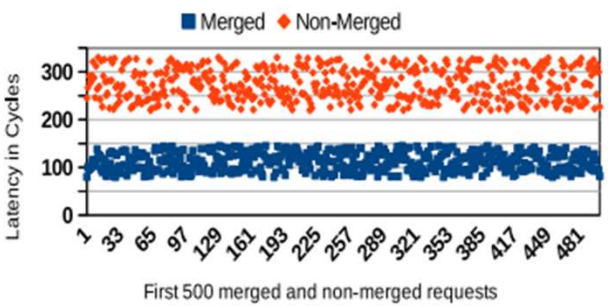
Latency observed by Spy (earlyReload)



Overlapping between Merged/Non-Merged significant

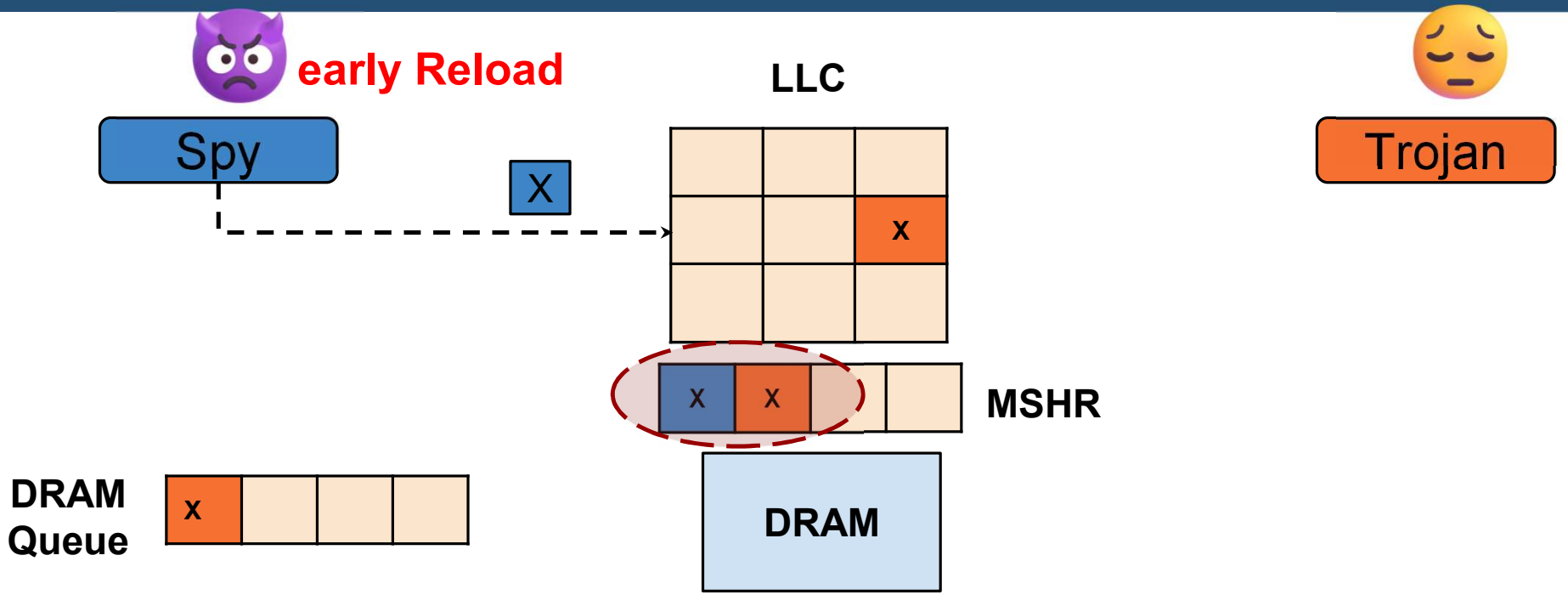


Overlapping between Merged/Non-Merged very less



No Overlapping between Merged/Non-Merged

Duplicate entry in MSHR



Observation: If the duplicate requests may pass through the MSHR, their chances of being merged in the main memory queue cannot be ignored. Bypassing the security-related responsibilities to the main memory cannot be considered a good solution

Advanced Randomized Caches

Countermeasure for LLC Timing-Channel Attacks

Cross-core Attacks

Randomisation

Randomize cache placement and replacement to make predictions harder.

Partitioning

Partition the cache ways or sets among cores to limit information leakage.

Shared Memory/Other

Request Wait

Introduce random delays in memory requests to obscure timing information.

Data Duplication

Duplicate data across multiple locations to eliminate contention-based leakage.

These techniques make cache behavior **less predictable**, reducing **information leakage** and **strengthening defense against** against LLC timing-channel attacks.

Conclusion

1 Cache Partitioning [1]:

- ***Static and Dynamic Partition.***
- ***Way-based and set-based partition.***

2 Cache Randomization [3, 4, 5]:

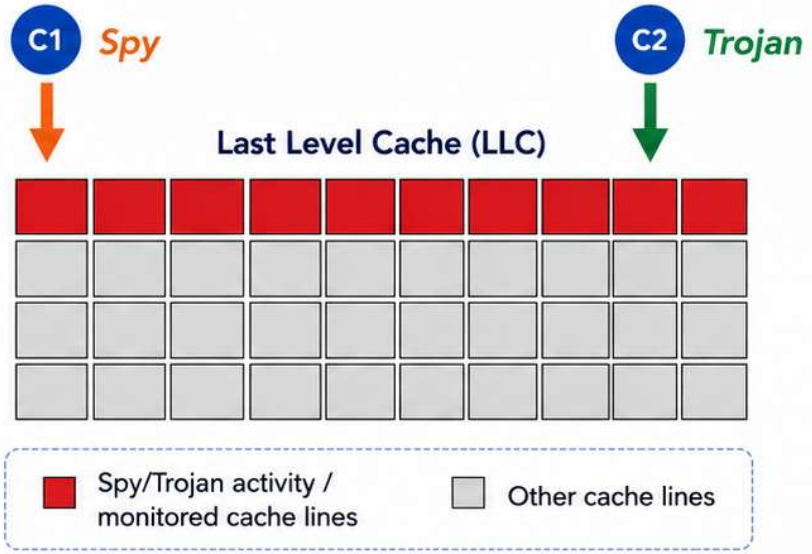
- ***On set-associative cache: CSEASER [3, 5]***
- ***On skew-associative cache: CEASER-S [4]***

3 Shared Address Space:

- ***Flush + Reload Attack***

4 Why further research in this area?

- Some of these techniques are not fully secure.
- Some of these techniques has high performance overhead.
- ***In some cases the performance is going down to what we archived before 2010.***



These techniques improve security, but there is always a trade-off between security and performance.

[1] J. Kaur and Shirishendu Das, "A survey on cache timing channel attacks for multicore processors," *Journal of Hardware and Systems Security*, 2021.

[3] M. K. Qureshi, "CEASER: Mitigating Conflict-Based Cache Attacks via Encrypted-Address and Remapping," MICRO, 2018.

[4] M. W. et. al., "ScatterCache: Thwarting cache attacks via cache set randomization," USENIX Security, 2019.

[5] M. K. Qureshi, "New attacks and defense for encrypted-address cache," ISCA '19.

THANK YOU!

Questions • Discussions • Insights

SECURE

EFFICIENT

PERFORMANT

RELIABLE

Building Secure and Efficient Systems for a Better Tomorrow